

A CLINICAL AI FIELD GUIDE

Claude Code *For Dummies*

*The clinical AI builder's guide to shipping software
with Claude as your pair programmer*

Contents

Introduction

PART I — GETTING STARTED WITH CLAUDE CODE

1. Meet Claude Code

2. Installing and Launching

3. Speaking Claude

4. Permissions and Safety

PART II — DAILY WORKFLOWS

5. Files, Tools, and the Terminal

6. Context Like a Pro

7. Models, Tokens, and Cost

8. Git Without Tears

9. Plan Mode and Thinking

PART III — MAKING CLAUDE CODE YOURS

10. CLAUDE.md — Your Project Instruction Manual

11. Slash Commands

12. Skills

13. Subagents and Agent Teams

14. Hooks

PART IV — PUSHING THE LIMITS

15. MCP Servers

16. Worktrees and Parallel Claudes

17. Headless Mode and Automation

18. Running for Hours

PART V — THE PART OF TENS

19. Ten Killer Workflows for Clinical AI Teams

20. Ten Rookie Mistakes to Avoid

21. Ten Slash Commands Worth Memorizing

22. Ten Things to Put in Your CLAUDE.md Today

APPENDICES

A. The Claude Code Cheat Sheet

B. Build Your Own Agent with the Claude Agent SDK

Introduction

So there you are, staring at a black terminal window with a blinking cursor, wondering how anyone is supposed to get useful work out of *this*. Somebody told you Claude Code was going to change how you build software — that it could write whole features while you slept, refactor legacy pipelines on a Friday afternoon, and free up half your week. And maybe they're right. But right now, all you see is a cursor.

This book is the bridge between that blinking cursor and the version of you who ships software the way modern engineers ship it: with an agent doing the keystrokes, and your brain doing the deciding.

About This Book

Claude Code For Dummies is a working clinician's guide to Anthropic's coding agent. It assumes you have a job to do — extract pathology from notes, build a Cockcroft-Gault calculator, plot some KDIGO trajectories, ship a Django dashboard for the chemotherapy unit — and that Claude Code is a tool, not a hobby.

The book is organized so you can read it cover to cover in a weekend, or jump to whichever chapter solves the problem in front of you. Each chapter is short. Each one ends with a "Try This" exercise and a "Watch Out" warning. Examples are drawn from real KHCC work — AIDI pipelines, clinical extractions, R analyses, Django apps — but every pattern works on any codebase.

We cover:

- **Part 1** — what Claude Code actually is, how to install it, how to talk to it, and how to keep it from doing anything you'll regret.
- **Part 2** — your daily workflow: files, context, money, git, and how to make Claude *think* before it codes.
- **Part 3** — the customization layer: `CLAUDE.md`, slash commands, skills, subagents, and hooks. This is where Claude Code goes from useful to indispensable.
- **Part 4** — pushing the limits: MCP servers, parallel worktrees, headless automation, and Ralph loops that run for hours while you sleep.
- **Part 5** — the *For Dummies* signature **Part of Tens**: ten clinical workflows, ten rookie mistakes, ten slash commands worth memorizing, ten lines for your `CLAUDE.md`.

Two appendices close the book: a one-page cheat sheet, and a short tour of the Claude Agent SDK for readers who want to build agents of their own.

Foolish Assumptions

I assume:

- You have a laptop running macOS, Linux, or Windows (WSL works fine).
- You can write a *little* code — enough to read a Python function and tell when it's wrong.
- You have an Anthropic account, or you're willing to make one. (Pricing is covered in Chapter 7.)
- You're allergic to wasted time.

I do **not** assume you've used a coding agent before, that you know what an MCP server is, or that you've ever lived in a terminal. If any of those things sound familiar, you'll move faster. If they don't — welcome. You'll be fine.

Icons Used in This Book

In the great tradition of the yellow-and-black tribe, this book uses four icons in the margins to flag the four kinds of paragraph that matter most.

 **Tip.** A shortcut, a keystroke, or a settings tweak that will save you time. Read these even when you're skimming.

 **Remember.** A concept you'll use repeatedly. If you forget the rest of the chapter, hold onto these.

 **Warning.** Things that have cost real people real money, real data, or real sleep. Read every single one.

 **Technical Stuff.** Optional plumbing. Safe to skip on first read; useful when you go to debug.

Beyond the Book

Two things travel with you:

1. **The cheat sheet** in Appendix A. Print it. Tape it to your monitor. Stop alt-tabbing.
2. A `CLAUDE.md` **starter** in Chapter 10. Copy it into every new project before you write a single line of code.

A Note on Style

You will sometimes catch the author being opinionated. Things like "don't use `--dangerously-skip-permissions` until you understand what you're allowing" or "do not let Claude touch the deceased-patient eval cohort." These are not arbitrary preferences. They are scars. They come from real

production pipelines at a real cancer center, where a silent failure can mean a patient missing from an AKI alert run. When you see a strong opinion in this book, it is there because somebody, somewhere, paid for it. You're inheriting the discount.

Where to Go from Here

If you've never opened Claude Code, **start at Chapter 1**.

If you've used Claude Code a little but feel like you're still typing into a void, **start at Part 3** — the customization chapters are where most casual users have left value on the table.

If you came to learn how to build long-running autonomous agents, **skip to Chapter 18 and Appendix B**.

If you just want the shortcuts, **flip to Appendix A**, then come back when something stops making sense.

Let's go.

PART I

Getting Started with Claude Code

Chapter 1: Meet Claude Code

You open a terminal, type one word, and a few minutes later fifty pathology reports have been read, graded, and dumped into a tidy CSV. And you didn't write a single line of code.

That is the trick, and that is the chapter.

Chatbots Talk. Claude Code Acts.

You already know what a chatbot does. You open ChatGPT or claude.ai in your browser, you type a question, it types back an answer. Useful. Sometimes brilliant. But fundamentally, a chatbot is a smart pen pal. It can describe what to do. It can't go do it.

Claude Code is not a chatbot. Claude Code is an **agent**. The difference matters more than any other concept in this book, so it is worth saying twice: a chatbot gives you advice; an agent takes actions.

You ask Claude Code to "read every pathology report in `./reports/`, pull out the histologic grade, and write the results to `grades.csv`." It does not respond with a Python tutorial. It opens the directory. It reads the files. It runs the extraction. It writes the CSV. Then it tells you it's done, and shows you the diff.

 **Remember.** Chatbots return text. Agents return changes to your filesystem. Everything else in this book follows from that one distinction.

This is the same Claude you already know: same model family, same training, same brain. What is new is the body. Claude Code is Claude with hands.

It Lives in the Terminal

Claude Code does not run in a browser. It runs in your terminal: that black-and-white window full of cryptic-looking text that you have probably seen developers use and politely ignored.

The terminal is not as scary as it looks. It is simply a way to talk to your computer by typing instead of clicking. And with Claude Code, you don't even need to learn the terminal. You only need to learn three things:

- How to **open** Claude Code: type `claude` and press Enter.
- How to **close** Claude Code: press `Ctrl+C` twice.
- How to **clear its memory** mid-session: type `/clear`.

That is genuinely the whole interface. Everything else is just plain English.

💡 **Tip.** If you have used VS Code, RStudio, or even just clicked around in a file explorer, you already know more than enough to use Claude Code. The terminal here is a delivery mechanism, not a skill you need to master.

What Claude Code Can Actually Do

If "takes actions" is too abstract, here is the concrete list of things Claude Code does for a living:

- **Reads files:** single files, whole directories, your entire repo.
- **Writes files:** creates new ones, edits existing ones, refactors across many at once.
- **Runs commands:** anything you can run in a terminal — `python` , `Rscript` , `pytest` , `dbx` , `git` , `npm` , `pip` , you name it.
- **Searches code:** by filename, by content, by pattern.
- **Hits the web:** fetches a documentation page, searches for an error message, pulls down a paper.
- **Manages git:** stages, commits, branches, pushes, opens pull requests.
- **Chains all of the above:** reads a file, sees an error, searches for the fix online, edits the file, runs the tests, commits the result.

In other words: Claude Code does the things you were going to do anyway, but faster and without typos. Your job moves up a level, from "how do I write this for loop" to "what should we actually be measuring here."

A Brief Tour of the Eight Built-in Tools

Under the hood, Claude Code has a small toolbox. You will hear these names again throughout the book, so meet them now:

Tool	What it does
Read	Opens a file and pulls its contents into context.
Write	Creates a new file (or fully overwrites one).
Edit	Makes a surgical change to a specific block of text inside an existing file.
Bash	Runs a shell command and returns the output.
Glob	Finds files by name pattern, e.g. <code>**/*.py</code> .
Grep	Searches <i>inside</i> files for a regex or string.
WebSearch	Looks something up on the live internet.
WebFetch	Pulls down the contents of a specific URL.

 **Technical Stuff.** These tools are implemented as structured function calls. When Claude decides to "read a file," it is really emitting a JSON payload that names the `Read` tool with a file path; the Claude Code harness executes the read and feeds the contents back. You will rarely need to think about this — but if you ever build your own agent with the Claude Agent SDK (Appendix B), this is the abstraction you will be working with.

You do not pick the tool. That is the magic. You describe a goal in plain English, and Claude decides which tool, or which sequence of tools, fits.

The Mental Model: Describe the Goal, Claude Picks the Tools

This is the one mental model that, once it clicks, makes Claude Code suddenly feel obvious.

You **describe the outcome**. Claude **chooses the path**.

Compare:

- *Old way:* "Use `os.listdir` to enumerate the reports folder, then loop over the files, then open each one with UTF-8 encoding, then write a regex for 'Histologic Grade:', then..."
- *New way:* "Extract the histologic grade from every report in `./reports/` and save the result as `grades.csv`."

You stopped giving instructions and started giving a destination. Claude handles the route.

 **Tip.** When a task feels stuck, ask yourself: am I describing the goal, or am I describing the steps? Stepping back to the goal almost always unsticks things.

A Clinical Example: Fifty Pathology Reports

Let's make this concrete with a job you might actually have on your desk this week.

You have fifty de-identified pathology reports sitting in `./reports/` as plain text files. You need the histologic grade out of each one for a small retrospective cohort. The grades are usually buried in a sentence like *"Tumor demonstrates moderately differentiated (Grade 2) features."* Sometimes the formatting is consistent. Often it isn't.

You open Claude Code in that project folder:

```
cd ~/projects/grade_extraction
claude
```

Then you type, in plain English:

Read every `.txt` file in `./reports/`. For each one, extract the histologic grade (1, 2, 3, or "unspecified"). Write a CSV called `grades.csv` with columns `filename` and `grade`. Use a small Python script. Show me the script before you run it.

Here is what happens behind the scenes:

1. **Glob** — Claude lists every `.txt` file in `./reports/`.
2. **Read** — Claude opens the first two or three to see what the reports look like.
3. **Write** — Claude drafts a short Python script and shows it to you.
4. **(You approve.)**
5. **Bash** — Claude runs the script.
6. **Read** — Claude opens `grades.csv` to confirm it looks right.
7. **Edit** — Claude notices three files where the regex missed and tightens the pattern.
8. **Bash** — Claude re-runs the script.
9. Claude reports back: 47 grades extracted, 3 reports flagged as "unspecified," CSV saved.

You watched the work happen. You read the script before it ran. You can rerun it tomorrow on a hundred more reports.

⚠ **Warning.** Even on a job this simple, do not run extraction on identifiable patient data in a folder that isn't approved for it. At KHCC, that means working inside Databricks or an approved AIDI environment, not on your personal laptop. The agent doesn't know the difference between de-identified text and PHI. You do.

💡 **Remember.** Claude Code does not replace your judgment about *what* to extract, *how* to validate it, or *where* the data is allowed to live. It replaces the typing.

Why This Changes Your Week

Here is what shifts once you internalize the agent model:

- The **boring half of your job** (boilerplate scripts, file munging, SQL scaffolding, regex tuning, ggplot tweaks) collapses from hours to minutes.
- The **hard half of your job** (deciding what the cohort definition actually is, whether the extraction is clinically valid, whether the model is fit for purpose) does not change at all. It is still on you. It always will be.

💡 **Tip.** The clinicians and data scientists who get the most out of Claude Code are not the strongest coders. They are the ones who can most clearly *describe what they want*. If you can write a good methods section, you can drive this tool.

Try This

Open a terminal in a directory that has a few plain-text files in it — any folder will do; a notes folder works fine. Type `claude` and press Enter. When the cursor appears, ask:

List every file in this directory, then tell me which file is the longest by line count.

Watch what Claude does. Notice that it uses `Glob` to find the files, `Bash` (or `Read`) to count lines, and then summarizes the result. You did not pick a single tool. You described a goal.

When it finishes, press `Ctrl+C` twice to exit.

Watch Out

Do not type a vague, sprawling request the first time you open Claude Code and then judge the tool by the result. *"Build me a clinical dashboard"* is not a prompt. It's a project. Claude will make something, and it will be the wrong thing, and you will conclude the tool is overhyped.

Start small. One folder. One job. One outcome. Build trust before you build a feature.

Chapter 2: Installing and Launching

Five minutes from now you'll have Claude Code installed, authenticated, and waiting for its first prompt.

The One-Line Install

Claude Code is distributed as an npm package. That means if you have Node.js 18 or newer, you have everything you need:

```
npm install -g @anthropic-ai/claude-code
```

That's it. One command. The `-g` flag installs globally, so `claude` becomes a command you can run from anywhere on your machine.

 **Tip.** Check your Node version first with `node --version`. If it says anything below 18, upgrade. The official Node site has installers for every platform, but if you already use a version manager (`nvm` , `fnm` , `volta`), use that instead. It's easier to swap versions later.

If you don't have Node at all yet, install it once and forget about it:

- **macOS:** `brew install node` (or download from nodejs.org)
- **Linux:** Your package manager (`apt install nodejs npm` on Ubuntu, `dnf install nodejs` on Fedora), or use `nvm`.
- **Windows:** Use **WSL2** (Windows Subsystem for Linux) and install Node inside Ubuntu. You *can* run Claude Code in native PowerShell, but WSL is closer to the environment every tutorial and skill is written against. Save yourself the headaches.

 **Warning.** Do not install Node as root with `sudo`. On a shared workstation or a Databricks driver node, that creates permission tangles you will spend an hour debugging later. Use `nvm` per user.

Authenticating

The first time you run `claude`, you'll be walked through one of two auth flows:

1. **Claude account login.** If you have a Claude Pro or Max subscription, sign in with your account. Usage runs against your plan. This is the option you almost certainly want.

2. **API key.** Paste an Anthropic API key. Usage is metered per token. Good for headless servers and CI. Bad for casual interactive use, because you'll spend more thinking about cost than coding.

The login is a browser handshake: Claude Code prints a URL, you open it, you click approve, and the terminal picks up where it left off.

 **Remember.** Most KHCC users want the Pro or Max subscription. The API key path is for the day you start running headless pipelines from a server (Chapter 17), not for your daily work.

Your First Session

Open a terminal. Go to a project directory. Type:

```
claude
```

A welcome panel appears, the cursor blinks, and you are inside. Type:

```
What is in this directory? Give me a one-paragraph summary.
```

Watch Claude run `Glob` and `Read`, then write you a short paragraph. Congratulations. You have used Claude Code.

The Three Keystrokes That Matter Most

You can use Claude Code professionally with only three commands. Memorize these and ignore the rest until you need them.

- `claude` : open a new session in the current directory.
- `Ctrl+C` **twice**: exit. The first press cancels whatever Claude is doing right now. The second press closes the session. (Pressing it once will *not* close Claude Code by itself, which is by design.)
- `/clear` : wipe the current conversation and start fresh, without leaving the session. Use this between unrelated tasks. Context rot is real.

 **Tip.** When in doubt, `/clear`. A clean session beats a messy three-hour one every time. Free advice from people who have lived a messy three-hour session.

`/help` and the Other Slash Commands

Inside Claude Code, anything starting with `/` is a built-in command. Type `/help` and you get the full list. The handful you'll meet in this chapter and the next:

- `/clear` : wipe the conversation.
- `/help` : list every slash command.
- `/cost` : show what this session has spent in tokens.
- `/model` : switch between Haiku, Sonnet, and Opus mid-session.
- `/permissions` : open the permissions panel (Chapter 4).
- `/statusline` : install a live HUD at the bottom of your terminal.

Don't try to learn all of them today. They will arrive one chapter at a time, in the order you actually need them.

Resuming a Session

You closed your laptop. You went to a meeting. You came back. Now what?

```
claude --resume
```

You'll see a list of every previous session, organized by project and timestamp. Pick the one you want. Claude reloads the full conversation history and you pick up exactly where you left off: same files in context, same plan, same opinions about your code.

 **Remember.** Sessions are saved automatically. You do not need to do anything to enable this. If you ever wonder "did I save that?", yes, you did.

This is how serious work actually happens with Claude Code. Almost nobody finishes a real project in one sitting. You build, you stop, you resume, you build, you stop. Treat sessions as resumable workspaces, not one-shot conversations.

The `cc` Alias Trick (Handle With Care)

Power users get tired of typing `claude` and even more tired of approving every action. There is a famous shortcut, from Suryansh Tiwari's "40 Practices," that you'll see all over the internet:

```
alias cc='claude --dangerously-skip-permissions'
```

Add that to your `~/.zshrc` or `~/.bashrc`, run `source ~/.zshrc`, and now `cc` launches Claude Code in YOLO mode, with no permission prompts on any action.

It is fast. It is also exactly what its name says.

⚠ Warning. Boris Cherny, the creator of Claude Code, has said publicly that he *never* uses `--dangerously-skip-permissions`. If the person who built the thing won't use that flag, you should think carefully before aliasing it as your default. Chapter 4 walks through the safer alternative, an allowlist in `settings.json`, that gets you most of the speed without the risk.

If you do want a shortcut alias, here is a less ruinous version:

```
alias cc='claude'
```

You save four keystrokes. You also still get to keep your filesystem.

The `/statusline` HUD

Once your basics are working, run this inside a Claude Code session:

```
/statusline
```

It installs a small shell script that displays a live status line at the bottom of your terminal after every turn: current directory, git branch, model in use, percentage of context window consumed. It is the closest thing Claude Code has to a dashboard, and it pays for itself the first time you avoid blowing past your context limit.

Installing on a Databricks Driver vs. a Personal Laptop

At KHCC, you are likely to install Claude Code in two very different places. The mechanics are the same. The expectations are not.

On a personal laptop:

- Standard `npm install -g`.
- Pro or Max subscription on your own KHCC SSO-linked Claude account.
- Use it for everything that doesn't touch patient data directly: local R analyses, dashboards, plot generation, building Django scaffolds, drafting policy documents, hammering on a `CLAUDE.md`.

On a Databricks driver node (or any shared compute):

- Install Node and Claude Code per-user, never as root.
- Authenticate with an **API key**, not interactive login. There is no browser on a driver node.
- Mount your code from a Databricks Repo, not from `/tmp`.

- Never paste patient identifiers into the conversation. Even on Databricks, the conversation transcript leaves the cluster.
- Treat the driver as a production-adjacent surface. The `aidi_catalog` is one accidental SQL statement away from a long Monday.

 **Warning.** On any shared environment touching real cohort data, install Claude Code in **plan mode** or behind an allowlist. The right default on a Databricks driver is "ask before everything." See Chapter 4.

 **Technical Stuff.** If you want Claude Code in a Databricks notebook cell instead of the driver shell, you can, but the experience is awkward. The terminal is the native habitat. Use `%sh` cells only for one-off `claude -p` headless calls (Chapter 17), not for interactive work.

Try This

Install Claude Code on your laptop right now. Authenticate. Then `cd` into any project folder and type:

```
claude
```

When the cursor appears, type:

```
/statusline
```

Approve the script when it asks. Then type `/help` and read the list of available commands once, slowly. Then `/clear`. Then exit with `Ctrl+C` twice.

You have now used six different controls in Claude Code. You are ahead of 80% of people who downloaded it.

Watch Out

Don't add `alias cc='claude --dangerously-skip-permissions'` to your shell config on day one. It is the single most common reason new users wreck a working repo in their first week. Build the habit of seeing permission prompts before you build the habit of skipping them. The flow you save is not worth the file you lose.

Chapter 3: Speaking Claude

Prompting Claude Code isn't a new language — it's your old language, used more carefully.

Plain English Beats Cleverness

There is no special syntax for Claude Code. No keywords. No XML tags you need to memorize. You type sentences. Claude reads them. Same as if you were briefing a junior analyst over Slack.

That's the good news. The catch is that "plain English" hides a wide gap between **a request a human can act on** and **a request an agent can act on**. The same words a colleague would charitably interpret will send Claude down the wrong road, because Claude isn't going to ask three clarifying questions before starting. It's going to start.

So the rule is simple:

 **Remember.** The quality of your output is almost entirely a function of the specificity of your prompt. Better prompts, better results. That is the whole game.

Bad Prompt vs. Good Prompt

Consider this prompt:

Build me a website.

A human collaborator would push back: *which website, for whom, in what language, with what content?* Claude Code, when asked this cold, will pick something and ship it. Probably a single-page React site with a fake "Our Mission" headline and Lorem Ipsum.

Now consider:

Build a one-page static landing page in plain HTML and CSS, no JavaScript framework. Audience is referring physicians for a chemotherapy preparation service at King Hussein Cancer Center. Sections: hero with a one-line value proposition, three feature cards (turnaround time, safety, formulary breadth), and a contact form that POSTs to `/api/contact`. Green and white color scheme. Mobile-responsive. Save it as `index.html` and `styles.css` in the current directory.

This is the same task. The second version takes ten times longer to type and saves you forty times the cleanup. Claude doesn't need you to be brief. It needs you to be specific.

 **Tip.** When in doubt, add constraints. Color, framework, file names, output format, audience, what *not* to do. Constraints don't slow Claude down; they steer it.

The Exception: Be Vague On Purpose When Exploring

There is one place where vagueness is a *feature*, not a bug — exploring code you don't fully understand.

If you've inherited a 4,000-line R script from a colleague who left and you have no idea what's in it, the best prompt is not a detailed one. It's:

Read `analysis.R` end to end. What would you improve in this file?

This gives Claude permission to surface things you wouldn't have thought to ask about. Hard-coded paths. Stale package calls. A loop that should be a `purrr::map`. A patient-level split that's accidentally a row-level split. Vagueness, used deliberately on unfamiliar territory, is a mapping tool.

 **Remember.** Be specific when you know what you want. Be vague when you don't. Both are valid. Sloppy is what happens when you can't tell which mode you're in.

Pointing Claude at Files With @

Inside a Claude Code session, typing `@` opens a file picker. You can also just type the path directly:

```
@scripts/extract_grades.py
```

When you reference a file with `@`, Claude reads it into context immediately. No searching the codebase, no token spent on `Glob` and `Grep`. You have told it exactly where to look.

This matters more than it sounds. Without `@`, a casual prompt like *"fix the bug in the extraction script"* sends Claude on a hunt — which directories matter, which file is the script, which function is the extraction. That hunt eats tokens and sometimes finds the wrong file. With `@scripts/extract_grades.py`, you save the search and you save yourself the wrong answer.

 **Tip.** When you know which files matter, name them with `@`. You will be right faster.

You can pile them up:

```
@sql/cohort.sql @notebooks/01_extract.py @notebooks/02_dose.py — walk me through how this pipeline assembles the AKI cohort, and tell me where the deceased-patient filter is applied.
```

That single prompt is worth ten minutes of "where is...?"

The `!` Prefix — Bash Output Straight Into Context

Sometimes the thing Claude needs to see isn't a file. It's the output of a command — a git status, a failing test, an error log, a row count. The `!` prefix lets you run a shell command and drop its output directly into Claude's context for the next prompt.

```
!git status
```

```
!python extract.py 2>&1 | tail -30
```

```
!Rscript -e 'sessionInfo()'
```

Now Claude can see exactly what you see. No paraphrasing. No "I think the error was something like..."

This pairs beautifully with the next idea.

Raw Data Beats Prose

This is one of the most under-appreciated rules of prompting an agent. **Don't describe the data. Show the data.**

Bad:

```
I'm getting a strange error in the extraction script — it says something about a missing column when I run it.
```

Good:

```
!python extract.py 2>&1 | tail -40
```

then:

Fix this.

You have just collapsed five back-and-forth turns into one. Claude sees the actual traceback, the actual filename, the actual line number. It edits the right file on the first try.

You can chain this from the shell too:

```
cat error.log | claude "explain this error and suggest a fix"
```

 **Tip.** If you find yourself typing a long paragraph that *describes* a bug, stop. Paste the log. Type "fix this." You will be done in half the time.

 **Warning.** Before you paste a log: scrub MRNs, names, and any other identifiers. Logs are notorious for capturing real patient data when something blows up mid-pipeline. The AIDI deny-list patterns from Chapter 4 don't help here — pasted text is pasted text. Look before you paste.

Voice Dictation Beats Typing

Run `/voice` inside Claude Code (where supported) and you get push-to-talk dictation. Hold a key, speak your prompt, release.

The interesting thing isn't speed. It's quality. When people speak a prompt out loud, they naturally include context, constraints, audience, and edge cases that they would never have bothered to type. Try it once and watch your own prompts get better.

 **Remember.** A spoken prompt is almost always a better prompt than a typed one. Type when you must. Talk when you can.

A KHCC Example: Bone-Only Metastatic Disease

Here is a real prompt shape that comes up constantly in AIDI work — extracting a specific clinical concept from a stack of radiology reports.

Bad prompt:

Extract bone metastasis from the reports.

This is ambiguous in at least four ways. *Which reports? What does "bone metastasis" mean — any bone involvement, or bone-only? What output format? What about uncertain or "probable" findings? Should you exclude post-treatment patients?*

Good prompt:

Read every `.txt` file in `data/radiology/`. For each report, decide whether the radiologist's impression supports **bone-only metastatic disease** — defined as: (a) at least one site of bone metastasis, AND (b) no evidence of visceral or nodal metastasis in the same study. Treat "no other sites identified" as supporting (b). Treat any mention of liver, lung, brain, peritoneal, or distant nodal disease as ruling out (b). For each report return one row in `bone_only.csv` with columns: `filename`, `bone_only` (TRUE/FALSE/UNCERTAIN), `evidence_quote` (the sentence you based the decision on). Use PydanticAI with the AIDI Azure OpenAI client (`gpt-4.1-mini`). MRNs in filenames are already Optimus-encoded — do not log raw names. Write the script to `notebooks/bone_only_extract.py`, but do not run it yet. Show me the script first.

That prompt is long. It is also unambiguous. Notice what it does:

- **Defines the clinical concept** ("bone-only" means bone AND not visceral/nodal).
- **Specifies the schema** (three columns, exact names).
- **Forces an evidence quote**, so you can spot-check.
- **Names the model and library** to use (so you don't get a random scikit-learn solution).
- **Reminds Claude of AIDI conventions** (Optimus, no raw PHI in logs).
- **Holds back execution** until you've reviewed the code.

 **Tip.** A useful prompting habit: write the prompt, then re-read it asking "could a smart but literal-minded reader interpret any sentence in two ways?" Every yes is a constraint you forgot to add.

A Short Checklist Before You Press Enter

When the prompt matters, walk through this list in your head:

- **What is the goal**, expressed as a deliverable? (A file, a CSV, a chart, a passing test.)
- **Which files** does Claude need to read? (Name them with `@`.)
- **Which conventions** should it follow? (Library, language, output format.)
- **What should it not do?** (Don't run the script yet. Don't touch the database. Don't change the prompts.)
- **How will you verify** it worked? (A row count, a sanity check, a known answer.)

Five questions. Thirty seconds. You will save half an hour.

Try This

Open Claude Code in any project. Pick one file you wrote recently — a Python script, an R analysis, a Quarto doc, a Django view. Try two prompts back to back:

1. *Vague*: "What would you improve in `@path/to/file`?"
2. *Specific*: "In `@path/to/file`, rename every variable that uses camelCase to snake_case, run the existing tests, and tell me which renames touched a public function."

Notice the difference in what comes back. Both are valid prompts — for different jobs.

Watch Out

Don't fall into the "I'll just clarify it later" trap. If your prompt is ambiguous, Claude won't stop to ask — it will pick an interpretation and start writing code against it. By the time you notice it picked the wrong one, you've spent ten minutes of session time and a few thousand tokens on the wrong answer. Spend the extra fifteen seconds on the prompt. Always cheaper than the correction.

Chapter 4: Permissions and Safety

There is exactly one chapter in this book that, if you skip it, will eventually cost you a file, a repo, or a Monday. This is that chapter.

Why Permissions Exist

Claude Code is not a chatbot. We hammered on this in Chapter 1, and it comes back to bite us here. Because Claude Code takes real actions on your real computer, it can also do real damage:

- Delete files.
- Overwrite working code with broken code.
- `git push --force` to main.
- Drop a table.
- Send an email to a real address.
- Hit a paid API a thousand times.
- Read an `.env` file containing real credentials and helpfully include them in a commit.

None of that is theoretical. All of it has happened to someone in the last six months.

So Claude Code ships with a permissions system. By default, before Claude does anything significant (write a file, run a shell command, hit the network) it stops and asks you. You see exactly what it wants to do. You say yes or no.

 **Remember.** The default permissions mode is "ask before doing anything that touches the world." That is the right mode to start in. Stay there until you understand what you're being asked to approve.

The Approval Tax

Default mode is safe, but after about an hour you notice the cost: Claude is interrupting you every fifteen seconds to ask permission for `npm run lint`, `pytest`, `git status`, and other things you'd be willing to bet your house on. This is the approval tax, and it kills flow.

The fix is not to turn permissions off. The fix is to **pre-approve** the things you trust, and keep gating the things you don't.

`/permissions` and the Allowlist

Inside Claude Code, type:

```
/permissions
```

You'll see a panel listing:

- **Allow:** actions Claude can take without asking.
- **Deny:** actions Claude is forbidden to take, ever.
- **Ask:** the catch-all default for anything not on either list.

You can add entries inline (Claude will offer to add the current pending command), or you can edit `.claude/settings.json` directly.

A reasonable starting allowlist for most projects:

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(git status)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(npm test)",
      "Bash(npm run lint)",
      "Bash(pytest:*)",
      "Bash(Rscript -e:*)",
      "Bash(ls:*)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Read(./credentials/**)",
      "Read(./**/*.key)",
      "Read(./**/*.pem)",
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)"
    ]
  }
}
```

 **Tip.** You don't have to write this JSON yourself. Inside Claude Code, just say: *"Add the common safe permissions to my settings — reading files, running tests, basic git commands, starting the dev server. Show me what you're adding before you save it."* Claude will draft the JSON and let you approve.

What belongs on each list?

Safe to allowlist (you do these a hundred times a day, and a mistake costs nothing):

- Reading files (`Read` , `Glob` , `Grep`).
- Running your test suite.
- Running your linter or formatter.
- Read-only git commands (`status` , `diff` , `log` , `branch`).
- Starting a local dev server.

Keep gated (the cost of a mistake is real):

- Installing packages.
- Deleting files or folders.
- Anything that touches the internet — API calls, web fetches, outbound HTTP.
- Anything that writes to a remote — `git push` , `dbx` , deployments.
- Anything that hits a database in write mode.

 **Remember.** The permissions file is checked into your repo. That means every teammate on the project picks up the same allowlist. Configure once, share automatically.

The Deny List Is Not Optional

The deny list is the other half of the system, and it matters more than the allowlist. The allowlist saves you keystrokes. The deny list saves you incidents.

Things that should be on the deny list in every single project, no exceptions:

- `Read(./env)` and every variant (`.env.local` , `.env.production` , etc.).
- `Read(./secrets/**)` .
- `Read(./credentials/**)` .
- `Read(./**/*.key)` and `*.pem` .
- `Bash(rm -rf:*)` .
- `Bash(git push --force:*)` and `Bash(git push -f:*)` .

Once a file is on the deny list, Claude can't read it even if you explicitly ask it to. The agent doesn't see the contents; it doesn't include them in search results; it can't accidentally paste them into a commit. This is the layer that turns a careless prompt into a non-event instead of an incident.

 **Warning.** A surprising number of `.env` files end up in conversation transcripts because a developer asked Claude to "help me debug why this isn't connecting" and Claude helpfully read the `env` file. Once it's in the transcript, it's in the model provider's logs. Put the deny list in place *before* you have that thought.

`--dangerously-skip-permissions` and Why Boris Doesn't Use It

There is a flag (you've seen it in Chapter 2) called `--dangerously-skip-permissions`. It turns off the permission prompts entirely. Claude does whatever it decides to do, with no human in the loop.

The flag is named exactly what it is. Boris Cherny, the creator of Claude Code, has said publicly that he never uses it himself. The recommended path, straight from the source, is to use a `settings.json` allowlist + the default `ask` mode for everything else.

 **Warning.** If the person who built Claude Code won't use `--dangerously-skip-permissions`, you should consider why. The flag is appropriate in *very* narrow circumstances (sandboxed CI jobs, ephemeral containers, Ralph loops in worktrees, Chapter 18) and inappropriate everywhere else. It is never the right default for interactive work on a repo you care about.

If you absolutely must run unattended (overnight refactor, long Ralph loop), pair the flag with **isolation**: a fresh worktree, or `--sandbox` (below). Never run it against your main checkout.

`--sandbox` Mode: The Safer "Run Wild"

Claude Code ships with a sandbox mode that uses OS-level isolation (Seatbelt on macOS, bubblewrap on Linux) to confine the session to a restricted filesystem and network view. From inside the sandbox, Claude can do whatever it wants; outside the sandbox, nothing changed.

```
claude --sandbox
```

This is the right way to let Claude experiment. Try a refactor. Run a wild prototype. Let it install seven npm packages you've never heard of. When you're done, review the diff in the sandbox and only carry over what you want.

 **Tip.** `--sandbox` + a worktree (Chapter 16) is the standard recipe for "I want to let Claude work autonomously without supervising." Two layers of isolation. Trivial to throw away. Real changes get explicitly merged back, never automatically.

Permission Modes

You can also set a session-wide mode without editing `settings.json` for every command. The most common modes:

- `default` : ask before risky actions, allowlist applies.
- `plan` : Claude can read and think and propose, but cannot write or run anything. Excellent for design-first work (Chapter 9).
- `acceptEdits` : auto-approve file edits, still ask before bash commands. The sweet spot for many developers.
- `bypassPermissions` : same as `--dangerously-skip-permissions` . Be careful.

Switch modes inside a session with `Shift+Tab` , or set a default in `settings.json` :

```
{
  "permissionMode": "default"
}
```

KHCC and AIDI: The Non-Negotiable Rules

Everything in this chapter applies double inside KHCC. Some rules that are not up for negotiation when working with AIDI data:

- **Never allowlist anything that touches `aidi_catalog`** . Every Databricks SQL statement against the catalog deserves a human eyeball, every time.
- **Never allowlist anything that touches `dbo.eval_runs`** or other shared eval tables. A typo costs a baseline.
- **Always deny `.env` , `*.key` , `*.pem` , `credentials/**`** . No exceptions, no project, no excuses.
- **Always deny `data/patients/**` , `data/raw/**` , and any path that could contain raw PHI**. Pasted or read, it doesn't matter. Treat the deny list as the wall.
- **Never use `--dangerously-skip-permissions` on a Databricks driver** or any node with line of sight to the catalog. Headless work against AIDI goes through reviewed, gated jobs, not interactive YOLO.
- **Never log MRNs or names in plaintext**. Optimus-encode MRNs. Fernet-encrypt names. This isn't a permissions issue; it's a habit. But your settings should at least *deny* reading the raw-name CSVs that might exist locally during a one-off pull.

⚠ Warning. A pipeline that silently skips a deceased patient because of a missing column is worse than a pipeline that crashes. The same logic applies to permissions: a permission check that silently auto-approves a destructive command is worse than one that interrupts you. When in doubt, gate it.

A starter `settings.json` for AIDI work looks roughly like this:

```
{
  "permissionMode": "default",
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(git status)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(pytest:*)",
      "Bash(Rscript -e:*)",
      "Bash(ls:*)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Read(./credentials/**)",
      "Read(./data/patients/**)",
      "Read(./data/raw/**)",
      "Read(./**/*.key)",
      "Read(./**/*.pem)",
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)",
      "Bash(git push -f:*)",
      "Bash(dbx deploy:*)",
      "Bash(databricks jobs run-now:*)"
    ]
  }
}
```

Adjust the allow list per project. The deny list is the floor — add to it, never subtract.

🔧 Technical Stuff. Patterns in `allow` / `deny` use a simple glob-like syntax. `Bash(git diff:*)` allows any `git diff` invocation; `Read(./env.*)` matches `.env`, `.env.local`, `.env.production`. When in doubt, test by typing the command in Claude Code and seeing what it asks for. The prompt will tell you the exact rule to add.

Try This

In any project you're already working in, run `/permissions` inside Claude Code and look at what's currently allowed. (If you've never configured this, the list will be empty.) Then ask Claude:

```
Draft a .claude/settings.json with a sensible allow list for read-only git commands and pytest,
and a deny list that blocks .env, secrets/**, credentials/**, *.key, *.pem, rm -rf, and
git push --force. Show me the file before saving it.
```

Read what it produces. Adjust to taste. Save it. Commit it.

Watch Out

Don't allowlist `Bash(*)`. People do this when the prompts get annoying. It defeats the entire system: you have just promoted Claude to root on your project. If you find yourself reaching for a wildcard allow, that's a signal that you should be in plan mode, not a signal that the allowlist needs to be looser. The constraint is the safety. Keep it.

PART II

Daily Workflows

Chapter 5: Files, Tools, and the Terminal

Every useful thing Claude Code does (reading a notebook, fixing a bug, grepping a repo, pulling a doc off the web) happens through a small set of built-in tools, and the sooner you stop thinking of them as magic the sooner you'll start steering them.

This chapter is the deep dive on those tools. Not a reference card (those live in Appendix A) but the working clinician's mental model of *which tool gets reached for when*, and how to nudge Claude toward the right one without micromanaging it.

The Toolbox

Claude Code ships with a handful of built-in tools. You'll see them flash by in the transcript as Claude works.

- **Read:** opens a file and pulls its contents into context. Line-numbered, partial reads supported, image-aware.
- **Write:** creates a new file (or overwrites one, if you let it).
- **Edit:** surgical string replacement on an existing file. This is what Claude reaches for 90% of the time when changing code.
- **Bash:** runs a shell command. `ls`, `git status`, `pytest`, `python script.py`, anything your terminal can do.
- **Glob:** fast file-pattern search. `**/*.py`, `src/extractors/*.sql`. Returns paths, not content.
- **Grep:** content search built on ripgrep. Regex-capable, fast, scoped by glob or filetype.
- **WebFetch:** pulls a URL and feeds its content to a sub-model that answers a question about it.
- **WebSearch:** a Google-style search that returns links and snippets.

 **Remember.** You don't pick the tool. Claude does. Your job is to describe the goal clearly enough that the right tool is the obvious choice.

When Claude Picks Which

There's a logic to it, and once you see it you stop being surprised.

- "What does this function do?" → **Read** on a known path, or **Grep** to find it first.

- "Find every place we call `optimus_encode` ." → **Grep**.
- "List all the SQL files in the extractions repo." → **Glob**.
- "Run the test suite." → **Bash**.
- "Add a column to this table definition." → **Read** then **Edit**.
- "Create a new notebook for the chemo prep extractor." → **Write**.
- "What does the KDIGO 2024 guideline say about Stage 3 timing?" → **WebSearch** then **WebFetch** on the best hit.

 **Tip.** If Claude reaches for Grep when you wanted it to just read a file, the fix is to *name the file*. Vague prompt = exploratory tool. Specific prompt = surgical tool.

The @ File Reference

This is the single most useful keystroke in Part 2 of this book.

Type `@` in the prompt and Claude Code opens a fuzzy file picker. Pick a file, and its path is dropped into your prompt with a sigil that tells Claude to read it *before* doing anything else.

```
Take a look at @notebooks/aki_extraction.py and tell me
why the baseline creatinine is null for MRN-encoded patients.
```

Without `@`, Claude might glob the repo, grep for "baseline creatinine," read three wrong files, blow 8,000 tokens, and *then* find the one you wanted. With `@`, it goes straight to the file.

 **Tip.** Use `@` ruthlessly. Every time you know which file matters, name it. Your context window is finite. Don't pay Claude to find what you already know.

You can `@` multiple files in one prompt:

```
Compare the staging logic in @notebooks/aki_v3.py and
@notebooks/aki_v4.py and tell me what changed.
```

The ! Bash Prefix

Less famous than `@`, just as useful.

Type `!` at the start of a line and the rest is run as a bash command, but instead of just printing the output, **the output is captured into context**. Claude sees it as if you had pasted it.

```
!git log --oneline -20
What changed in the eval pipeline in the last two weeks?
```

Or:

```
!python -c "import polars as pl; print(pl.read_parquet('data/cohort.parquet').schema)"
Now write the extraction prompt assuming those columns.
```

 **Technical Stuff.** Under the hood, `!` is just a Bash tool call where the output is teed straight into the next prompt. It's a shortcut for "run this, then think about the result." You can do the same thing in two messages. `!` saves you the round trip.

Show, Don't Tell: Pasting Screenshots

Claude Code is multimodal in the terminal. You can paste an image directly into the prompt (a Databricks error screenshot, a chart that looks wrong, a slide from a CAP protocol PDF) and Claude will read it.

On Mac, `⌘V`. On Windows, `Ctrl+V`. WSL users, your terminal emulator needs to support it (Windows Terminal does, most do now).

 **Tip.** For visual bugs (a Plotly chart with the wrong axis, an R Markdown output with a busted gtsummary table) screenshotting and pasting is *always* faster than describing the problem in words. Show, don't tell.

The image goes into context like any other source. It costs tokens (vision tokens are pricier than text), so don't paste a screenshot of something Claude could read as text.

Bash Directly, or Through Claude?

This is the question that separates the tourists from the residents.

Run it yourself when: - It's a one-liner you've typed a thousand times. `git status`. `ls -la`. `du -sh`. - It's destructive and you want zero ambiguity. `rm -rf node_modules` is your finger on the trigger, not Claude's. - You want the output for *yourself*, not for Claude. (Why pay tokens to put `ps aux` in context?)

Let Claude run it when: - The output is going to inform the *next* thing Claude does. - It's a multi-step pipeline that benefits from Claude reasoning about intermediate output. - You want it logged in the transcript so future-you can see what happened.

 **Warning.** Do not let Claude run `rm`, `git reset --hard`, `git push --force`, or any destructive operation without reading the diff first. Permissions can be set to gate these (see Chapter 4). The default settings protect you. Custom settings can un-protect you. Be deliberate.

A KHCC Walkthrough: Pointing at One Notebook

Say you're working on the AKI alerting pipeline. The `aidi-extractions` repo has 80 notebooks. You want to fix a bug in one.

The slow way:

```
Find the AKI extraction code and fix the bug where baseline
creatinine is sometimes null.
```

Claude will glob, grep, read four files, build a mental model of the repo, then maybe find it. Easily 10,000 tokens before any actual work happens.

The fast way:

```
@notebooks/aki_extraction.py – the baseline_creatinine
column is null for ~12% of patients in the last run.
Read the function that computes it and tell me why before
proposing a fix.
```

Now Claude reads exactly one file, has the function in context, and starts reasoning about the bug. Maybe 2,000 tokens to a hypothesis.

The difference compounds. Over a workday of twenty prompts, the `@` habit saves an hour and tens of thousands of tokens.

 **Remember.** Tokens are time *and* money *and* signal-to-noise. Every token Claude reads that wasn't necessary is a token of noise crowding out the thing you actually wanted it to focus on.

When Glob and Grep Earn Their Keep

`@` is right when you know the file. When you don't, Glob and Grep are your friends.

```
Grep the aidi-extractions repo for any call to
azure_openai_client. I want to know which pipelines still
use the old wrapper.
```

That's a Grep job. Claude will pull back every match with line numbers, and you can decide which ones matter.

```
Glob every R Markdown file under analyses/ that was modified
in the last 30 days.
```

That's Glob plus a tiny bit of Bash. Don't fight the tool.



Technical Stuff. Grep here is ripgrep under the hood: fast, regex-aware, respects `.gitignore`. If you're used to BSD grep, the syntax is mostly the same but a few flags differ. Claude knows. You don't have to.

WebFetch and WebSearch: Use Sparingly

These pull external content into context. They're great for:

- Looking up a library's current API ("does pandas 2.2 still ship `read_clipboard` ?").
- Pulling a guideline you don't have locally (a specific CAP protocol page, a KDIGO update).
- Checking whether a specific error message has a known fix.

They're terrible for:

- Replacing documentation you should have downloaded once and put in the repo.
- Random fact-checking that doesn't change the code.
- "Just look around the internet a bit." You'll burn context with nothing to show.



Warning. Never WebFetch a page that might contain credentials, internal URLs, or patient data. The page contents land in your transcript. Treat WebFetch like any other external read: it leaves a trace.

The Mental Model

Here's the whole thing in one sentence.

Claude has a small set of hands. Your prompts decide which hand reaches.

@ says *use the Read hand on this file*. ! says *use the Bash hand on this command, then think*. A vague prompt says *figure out which hand to use, and good luck*. Specificity does the steering.

Try This

1. Open Claude Code in any project. Type `@` and watch the file picker appear. Pick a file. Ask "summarize this file in three sentences." Notice how the first thing Claude does is Read, with no exploration.
2. Run `!git log --oneline -10` and follow with "what's the theme of the last ten commits?" Confirm Claude has the log in context without you having to paste it.
3. Take a screenshot of any error message in your editor or a Databricks cell. Paste it into Claude Code and ask "what's wrong here?" Notice how much faster this is than typing the error out.
4. Ask Claude to "find every place we use the old Fernet wrapper" and watch which tool it picks. Then ask the same thing but say "grep for `legacy_fernet` ." Compare the cost.

Watch Out

- **Don't let Claude grep what you can `@` .** Naming the file is always cheaper than searching for it.
- **Don't paste screenshots when text would do.** Vision tokens are expensive. Use images for things that are genuinely visual.
- **Don't let Claude run destructive Bash without reviewing.** Read the command before you approve. Permissions are not a substitute for paying attention.
- **Don't WebFetch sensitive pages.** Treat the web tools like any external network call. Assume the response lands in your transcript and your logs.
- **Don't fight the tool selection.** If Claude consistently reaches for Grep when you wanted Read, the prompt is the bug, not the agent.

Chapter 6: Context Like a Pro

The single biggest reason Claude Code gives you a bad answer is not that the model is dumb. It's that the context window is dirty.

This chapter is about how to keep it clean.

What the Context Window Actually Is

Think of it as Claude's short-term memory. Everything Claude can "see" right now lives inside one fixed-size buffer: the system prompt, the `CLAUDE.md` file, the conversation so far, every file Claude has Read, every tool result, every screenshot. When that buffer fills up, things start falling out or getting fuzzy.

The numbers, in 2026:

- Sonnet 4.6: 200K tokens default, **1M tokens** if you pop `/model opus[1m]` or `/model sonnet[1m]`.
- Opus 4.6: same, 1M-token mode available.
- Haiku: smaller, but still generous.

 **Remember.** A million tokens is roughly 750,000 words. That's eight novels of memory. You will not run out by accident. You *will* run out by being sloppy.

The Green Bar (and What It's Trying to Tell You)

At the bottom of the Claude Code TUI there's a status line, and on it there's a thin bar — usually green, sometimes yellow, sometimes red. It tracks how full your context window is right now.

- **Green:** you have plenty of room. Keep working.
- **Yellow:** past halfway. Start being intentional about what you load.
- **Red:** close to the auto-compact threshold (~85%). Either compact now on your terms, or let the system do it on its terms.

 **Tip.** Glance at the green bar at the end of every prompt. It's a free signal. Most people ignore it until things go sideways.

Context Rot: The Symptoms

You'll know it when you feel it.

- Claude forgets a constraint you stated ten messages ago ("I told you not to touch `prompts.py`").
- Answers become more generic and less grounded in your actual code.
- Claude re-reads files it has already read.
- Suggestions contradict earlier ones.
- Tool selection gets erratic.

These are the symptoms of a context window full of stale and conflicting information. The model isn't broken. The conversation is.

The `/clear` Reflex

`/clear` wipes the conversation and starts a fresh session. Same `CLAUDE.md`, same project, same permissions, but no memory of what just happened.

The Suryansh rule (tip #13 from his "Claude Code tips" list, and it's the right rule): **between unrelated tasks, `/clear`.**

You just finished refactoring the AKI baseline-creatinine logic. You're about to start writing a Press Ganey survey parser. These tasks share *zero* context. Carrying the AKI conversation into the new task buys you nothing and costs you signal.

```
/clear
```

That's the whole command. One word. Use it more than you think you should.

 **Remember.** `/clear` is not destructive. Your code is on disk. Your `CLAUDE.md` is on disk. Your git history is on disk. The only thing you lose is the chat, which was about to start rotting anyway.

The Two-Failed-Corrections Rule

This one is tip #14 from the same list, and it has saved me hours.

If Claude has failed to fix the same bug twice in a row, stop. Don't try a third time in the same context. Start fresh.

Why? Because by the time Claude has flailed at a bug twice, the context contains two wrong hypotheses, two failed diffs, and two rounds of apologies. The third attempt is fighting *that noise* as much as the actual bug. A clean context with a sharper prompt ("the bug is in `compute_baseline()`, the failure mode is null returns for MRNs starting with K, here's the function") will out-perform the third correction every time.

 **Warning.** "Just one more try" in a polluted context is the surest way to burn an hour and not ship anything. Two strikes, you clear.

`/compact` : The Surgical Option

Sometimes you don't want to throw the conversation away. You just want it slimmer. That's what `/compact` is for.

`/compact` asks Claude to summarize the conversation so far, drop the raw tool calls and file reads, and keep going with the summary in place of the verbose history.

The trick is the optional argument. You can *guide* the summary:

```
/compact focus on the API changes and the failing test cases
```

```
/compact keep the column schema and the eval results, drop everything else
```

 **Tip.** A guided `/compact` is dramatically better than an unguided one. Tell the summarizer what matters. It's not a mind reader.

Auto-compaction triggers at around 85% of the context window: same mechanism, no guidance, takes its best guess. If you have a long session going and you know what matters, run a guided `/compact` *before* the auto-compact fires, so you control what survives.

`/rewind` and Esc-Esc: Time Travel

Sometimes the problem isn't memory. It's that Claude went down a wrong path five edits ago and you want to undo.

`/rewind` opens a list of every prompt in the session and lets you jump back to any of them. You can choose to restore *just the code*, *just the conversation*, or *both*. The conversation snapshots are taken at every turn; the code snapshots are taken before every edit.

Esc-Esc is the keyboard shortcut for the same thing.

```
Esc Esc
[pick a point in the list]
[choose: code only / conversation only / both]
```

 **Technical Stuff.** Checkpoints are stored locally per session. They're separate from git. You can rewind a file Claude touched even if you never committed. They're cheap insurance.

 **Tip.** When Claude does something destructive and weird, your reflex should be Esc-Esc, not panic. Rewind first, diagnose second.

The 1M Token Context: When It's Worth It

`/model opus[1m]` (or `/model sonnet[1m]`) switches you into the 1M-token context mode. It's not free (bigger context costs more per token) but for the right task it's a different tool.

When 1M earns its keep:

- Reading a giant codebase to plan a refactor.
- Cross-referencing a long protocol document (a 200-page CAP PDF, a full clinical guideline) against your extraction code.
- A multi-hour analysis session where you can't tolerate auto-compacts dropping detail.

When it doesn't:

- Daily prompt iteration. 200K is plenty.
- Quick fixes. You'll never get past 5%.
- Batch jobs over thousands of small inputs. Haiku in CLI mode is cheaper.

 **Warning.** A bigger context isn't a substitute for a clean context. Bad signal at 1M is worse than good signal at 200K. Don't reach for the big context as a way to avoid thinking.

A KHCC Walkthrough: 200 Pathology Notes

You're running a pathology extraction over 200 notes. Each note is 1–3 pages. You're iterating on the prompt as you go.

Here's the rhythm.

Note 1–10: Iterate freely. Look at outputs, tweak the prompt, re-run. Context fills with examples and failure modes. This is good. You want it in context while you're tuning.

Around note 30 (green bar yellow): Run `/compact` focus on the prompt iterations and the three remaining failure modes. Drop the raw note text and the successful extractions. Keep what matters for further tuning.

Note 60 (you've nailed the prompt): `/clear`. The prompt is now stable, lives in a file, and you don't need the tuning history anymore. Start fresh for the bulk run.

Bulk run, notes 1–200: Don't do this conversationally. This is a CLI-mode (`claude -p`) job. The 200 notes don't belong in your interactive context window at all. Each note gets its own invocation, its own clean context. (More on this in Chapter 16.)

If extractions start drifting halfway through: You're not seeing context rot. There is no context. Each note is fresh. If quality drops, it's the prompt or the model. Not memory.

 **Remember.** Long batches don't belong in interactive context. Interactive context is for *tuning*. CLI mode is for *running*. Mixing them is the single most common waste of money I see.

When to Start a Brand New Session

A new session (just close Claude and run `claude` again, or `/clear` if you want to stay in place) is the right move when:

- You're switching projects or repos.
- You're switching tasks that share no files.
- The conversation has been wrong twice in a row.
- The green bar has been yellow for a while and a guided compact would lose too much.
- You want to test "is this a context problem or a real bug?" Clearing is the diagnostic.

 **Tip.** When you finish a chunk of work, take 60 seconds to write what you accomplished into a markdown file in the repo *before* you clear. Future-you will be able to resume by reading the file. Don't rely on the chat to remember.

Try This

1. Open a long-running Claude Code session. Run `/compact` focus on [the thing you care about] and read the summary. Notice what survived and what didn't.
2. Make Claude do something wrong on purpose ("change this file to break the import"). Then Esc-Esc and roll it back. Confirm the file is restored without touching git.
3. Switch to 1M context with `/model opus[1m]`. Run `/cost` (we'll dig into this next chapter) and compare. Decide whether the extra capacity is worth the price for what you're doing.

4. Next time you catch yourself on a third "actually, the problem is..." correction, stop, `/clear` , and write a sharper prompt. Notice how much faster the third attempt resolves than it would have on the polluted context.

Watch Out

- **Don't ignore the green bar.** Yellow is your cue to compact. Red is your cue to be embarrassed you didn't compact already.
- **Don't `/compact` without a focus hint.** Unguided summaries lose the thing you actually cared about.
- **Don't run batch jobs in interactive context.** That's what CLI mode is for. Mixing them wastes money.
- **Don't rely on the chat as memory.** Anything you'll need tomorrow goes in a file. The chat is short-term memory only.
- **Don't switch to 1M context as a band-aid.** A bigger room doesn't clean itself. Compact and clear first.

Chapter 7: Models, Tokens, and Cost

Claude Code isn't one model. It's a family, and using the wrong family member for the job is how you accidentally spend \$80 on a typo fix.

This chapter is about picking the right model, understanding what you're paying for, and knowing the difference between a subscription that saves you money and an API key that quietly drains it.

The Family

Three models, three personalities, three price points.

- **Haiku:** fast, cheap, light. Good at classification, extraction with a tight schema, structured rewrites, batch jobs over thousands of inputs. Not so good at reasoning through ambiguity, multi-file refactors, anything where "judgment" matters.
- **Sonnet:** the default. Smart, fast enough, priced for daily use. This is the model you'll spend 80% of your time with. Good at everything. Great at the kind of mixed reading/writing/debugging that fills your workday.
- **Opus:** the smartest one. Slower. More expensive. Worth it for hard architectural decisions, gnarly SQL that has to be right the first time, multi-step planning, anything where a wrong answer costs you an hour.

 **Remember.** The model is a dial, not a religion. Sonnet for most things, Opus when it matters, Haiku when the task is small and repetitive. Match the model to the task, not the task to the model.

Switching Mid-Session

You don't commit to a model when you open Claude Code. Switch any time with:

```
/model
```

That pops a picker. Or be explicit:

```
/model opus  
/model sonnet  
/model haiku  
/model opus[1m]
```

The `[1m]` suffix asks for the 1M-token context window (covered in Chapter 6). It's only available on Sonnet and Opus.

 **Tip.** A common workflow: start in Sonnet, hit a wall on a hard problem, `/model opus`, get past the wall, `/model sonnet` to keep iterating cheaply. The hard part doesn't have to cost premium for the whole session.

What's a Token, Really?

A token is roughly $\frac{3}{4}$ of a word in English. "Patient" is one token. "Patients" is one token. "Pneumocystis jirovecii" is probably four tokens. Code tokenizes differently again: a Python `def` is one token, an obscure variable name might be three.

The numbers you want to keep in your head:

- **1,000 tokens $\approx$ 750 words $\approx$ one short page.**
- A typical Claude Code prompt: 50–200 tokens.
- A typical response with a small edit: 500–2,000 tokens.
- A typical file Read: 500–5,000 tokens.
- A massive PDF: 50,000+ tokens.
- A 1M-token window: enough to hold a small book and still have room to work.

 **Technical Stuff.** Input tokens and output tokens are priced differently: output is more expensive. Cached prompt prefixes (the `CLAUDE.md`, the tool definitions, the system prompt) are billed at a steep discount on cache hits. The TUI handles caching automatically; you don't tune it.

`/cost` and `/stats`

Two commands. Memorize both.

`/cost`: what you've spent in *this session*. Total dollars, broken down by model. You'll see lines like "Sonnet 4.6: 142,000 tokens in, 18,000 tokens out, \$0.83." Run it before and after a big task to know what that task actually cost you.

`/stats`: a richer view. Shows tokens by model, cache hit rates, the breakdown of input vs. output, and (depending on version) a running total across recent sessions.

 **Tip.** Run `/cost` at the end of every working day for a week. You'll quickly learn which tasks are cheap and which are expensive. After that, your sense of "is this prompt going to cost a quarter or four dollars" gets accurate, and you'll stop over-thinking small spend.

Pro, Max, or API?

Three ways to pay. Pick the one that matches your *actual* usage, not the one that sounds impressive at dinner.

Claude Pro (\$20/month)

Generous for the hobbyist or the clinician who codes for an hour a day. All models included. Predictable. Tells you when you're rate-limited, doesn't surprise-bill you. If you're not sure whether you'll stick with Claude Code, this is where to start.

Claude Max (\$100/month or \$200/month)

For people who live inside Claude Code. The \$100 tier is enough for a serious solo developer working full-time. The \$200 tier ("Max 20×") is for the heavy operator: multiple sessions in parallel, agent teams, big context windows, long sessions. Both include all models, both rate-limit before they bill you.

 **Tip.** If you find yourself doing math like "can I afford to switch to Opus for this?", you're on the wrong plan. Max exists so you stop counting.

API (pay-as-you-go)

You bring an Anthropic API key, Claude Code uses it, you pay per token. No subscription, no rate limits beyond the API's own quotas.

When the API wins:

- Very light usage. A few sessions a week, mostly Haiku/Sonnet. You'll pay \$5–20/month and beat any subscription.
- Headless / CLI-mode automation. Scheduled jobs, Ralph loops, batch extractions. These don't benefit from a Max plan's interactive limits; they benefit from per-token billing.
- You're a team and you want centralized billing across many users via Anthropic's API.

When the API loses:

- Daily interactive coding. You'll blow past \$200/month easily and have no rate limit telling you to slow down.

 **Warning.** The API has *no soft warning* before it gets expensive. A bug in a Ralph loop can chew through \$100 in an afternoon if you're not watching. Set spending caps in the Anthropic console. Treat your API key like a credit card.

"Ultrathink" and the Old Magic Words

A bit of history is owed here.

In earlier versions of Claude Code, you'd type words like `think`, `think hard`, `think harder`, or `ultrathink` in your prompt to nudge the model into spending more reasoning tokens before answering. Each word allocated a bigger thinking budget.

On Opus 4.6 (and Sonnet 4.6), this is largely automatic: extended thinking is on by default, with an adaptive budget. The model decides how much to think based on the difficulty of the prompt.

But the keywords still work, and on hard problems they still help. `ultrathink` on Opus 4.6 cranks the reasoning budget to its maximum, and you'll feel it on problems like:

- "Plan a refactor of the AKI staging logic so we can swap KDIGO versions without rewriting the alert pipeline."
- "Here are three SQL queries that should give the same answer but don't. Find the difference."
- "Design the schema for an evals table that supports retries, partial matches, and frozen ground truth."

 **Technical Stuff.** Thinking tokens are billed like output tokens. `ultrathink` on Opus can mean tens of thousands of extra tokens before the model writes a single word of response. Worth it for the right problem. Wasted on "rename this variable."

A KHCC Walkthrough: Picking Models Through a Workday

Morning. You're iterating on the chemo prep extractor prompt. Twenty cycles of "tweak prompt, run on 5 examples, look at output, tweak again." This is **Sonnet** work. Cheap, fast, perfectly capable.

11am. You hit a snag: the SQL that pulls the cohort is double-counting patients with multiple admissions in the same week, and you've been staring at it for fifteen minutes. `/model opus`, paste the query, `ultrathink - explain why this is double-counting and propose a fix that doesn't break the multi-admission case`. Five minutes later, you have it.

```
/model sonnet
```


Back to iterating.

Afternoon. The prompt is locked. You need to run it over 4,000 historical notes for the validation set. This is not an interactive job. You write a small Python script that calls the Anthropic API directly with **Haiku** for each note. Haiku is plenty for structured extraction once the prompt is solid, and at Haiku prices, 4,000 notes costs you a few dollars instead of several hundred.

 **Remember.** Sonnet for tuning. Opus for the hard moments. Haiku for the batch. Three models, three jobs. Don't pay Opus prices to do Haiku work, and don't ask Haiku to plan your architecture.

A Word on Cost Discipline

The dollar amounts in this chapter sound small until they aren't.

A solo clinician using Claude Code an hour a day on Sonnet might spend \$30–50/month on the API, or get the same usage covered by a \$20 Pro plan.

An AIDI engineer running multiple sessions, Opus for hard tasks, occasional 1M-context dives, agent teams, and a few automated CLI jobs in the background can easily spend \$400–800/month on the API, and would be far better off on Max 20×.

A Ralph loop misconfigured to keep retrying a failing eval forever can spend \$200 overnight on the API. Don't ask me how I know.

 **Warning.** Two rules. (1) Set a hard spending cap on any API key Claude Code uses. (2) If you're spending more than \$200/month on the API for interactive work, you're on the wrong plan.

Try This

1. Run `/cost` right now, in your current session. Note the number. Run it again at end of day. You'll be calibrated within a week.
2. Find the most recent prompt where you felt Claude "didn't quite get it." Switch to `/model opus`, paste the same prompt with `ultrathink`, and compare. Decide whether the answer was worth the price.
3. If you have an API key, log into the Anthropic console and set a monthly spending cap. Pick a number that would *hurt* if you hit it accidentally. Then build the habit of staying well under.

4. Try one batch job in Haiku. Pick a task you'd normally do in Sonnet (classifying a few hundred lines of text, extracting fields from a list of forms). Run it once in Sonnet, once in Haiku. Compare quality and cost. Decide for yourself.

Watch Out

- **Don't default to Opus for everything.** It's a screwdriver, not a hammer. Most of your work is Sonnet work.
- **Don't run batch jobs on a Max plan thinking it's "free."** Max has rate limits. Hit them, and your interactive sessions stall.
- **Don't use the API without a spending cap.** A bug in a loop will out-spend a Max plan in a single afternoon.
- **Don't paste secret keys into Claude Code thinking the cost is the worry.** It isn't. The keys land in your transcript and possibly your logs. That's a security problem, not a billing problem.
- **Don't switch to 1M context just because you can.** It costs more per token, and a bigger room doesn't help if you can't keep it tidy (re-read Chapter 6).

Chapter 8: Git Without Tears

If you've ever rage-quit a `git rebase`, this chapter is for you. Claude Code can handle most of the git plumbing you used to dread, leaving you to do the part that actually requires a human: reading the diff.

What Claude Does on Your Behalf

Out of the box, Claude Code knows git. Ask it to:

- Stage files.
- Write a commit message.
- Commit.
- Push.
- Pull.
- Branch.
- Diff.
- Show recent history.
- Open a PR.

...and it does, by running the same `git` commands you'd type, in the same shell. No magic. No proprietary wrapper. Just `git` through the Bash tool.

```
Stage the changes in @notebooks/aki_extraction.py and write
a commit message that explains why the baseline computation
changed.
```

Claude will run `git diff`, read the change, draft a commit message that actually reflects the diff, then `git add` and `git commit`. You review the message before it commits. Claude shows you the diff and the proposed message in the same turn.

 **Remember.** Claude doesn't know your repo's *intent*. It knows the diff. The commit message it writes is only as good as the diff explains itself. If your change is half-finished and Claude writes a vague message, the right fix is finishing the change, not retitling the commit.

The Commit Message Habit

Bad commit messages are a project-wide tax. "fix bug." "update code." "wip." Six months from now, when you're tracing why the AKI alerts started double-counting, those commits are useless.

Claude writes better commit messages than most humans, *if* you let it look at the diff. The recipe:

```
Look at the staged diff. Write a commit message that
follows our convention (subject under 70 chars, body
explaining the why, not the what). Don't commit yet – show
me the message first.
```

Claude shows you the message. You read it. You either say "good, commit" or you tweak the message and *then* commit.

 **Tip.** Put your commit-message style in `CLAUDE.md` once. "Use conventional commits. Subject under 70 chars. Body wraps at 72. Explain the why." Claude will follow it forever after.

`gh` Is the PR Friend

For anything that touches GitHub (pull requests, issues, reviews, releases), Claude Code prefers the `gh` CLI over any MCP integration.

Why? Two reasons:

1. **Context efficiency.** `gh` returns clean, scoped text. A GitHub MCP server returns a lot of metadata you didn't ask for, and that metadata lands in your context window.
2. **It's already on every developer's laptop.** `gh` is GitHub's official CLI. If you have git, you can have `gh`.

```
gh pr create --title "fix: baseline creatinine null for K-prefix MRNs" --body "..."
```

Claude will run that for you. You don't memorize the flags.

 **Technical Stuff.** If you have a GitHub MCP server installed *and* `gh` available, Claude tends to reach for `gh` first because it's lighter. You can pin this behavior in `CLAUDE.md`: "For GitHub operations, always prefer `gh` over MCP."

Opening a PR

The whole flow, in one prompt:

```
Push the current branch, open a PR against main, and write
a description that summarizes the three commits I just
made. Title under 70 chars. Don't mark it ready yet –
open it as draft.
```

Claude will:

1. `git push -u origin <branch>` (set upstream if needed).
2. `git log main..HEAD` to read the commits.
3. `gh pr create --draft --title "... " --body "... "` with a body that actually reflects what changed.

You review the PR URL in the terminal, click it in your browser, and finish the description if needed.

 **Tip.** Ask Claude to include a "Test plan" checklist in the PR body. Even on solo projects, future-you appreciates the breadcrumbs.

Reviewing the Diff Before You Approve

This is the load-bearing habit of working with an AI on git.

Always read the diff before you say "commit."

```
Show me the diff for the staged changes before we commit.
```

Claude prints `git diff --staged`. You read it. You catch:

- The print statement Claude left in for debugging.
- The unrelated whitespace change that crept into a file Claude was just supposed to read.
- The hardcoded path that should have been a variable.

If you don't review, none of those land in CI. They land in `main`. And then you spend Friday afternoon reverting.

 **Warning.** It is faster to read a diff than to revert a commit. Always. No exceptions.

The Checkpoint System (and Why Git Still Matters)

Claude Code has its own session-level checkpoint system: every file edit creates a local snapshot, and `/rewind` (or Esc-Esc) lets you roll back without touching git. We covered this in Chapter 6.

But checkpoints are **local, session-scoped, and ephemeral**. They live in your `~/.claude` directory. They go away.

Git is the *durable* record. Checkpoints are for "oh god, undo the last fifteen minutes." Git is for "this work needs to survive a laptop crash and be reviewable by my team six months from now."

 **Remember.** Checkpoint first, commit second. Use checkpoints liberally (they cost you nothing). Use commits deliberately (each one is a permanent record).

Worktrees: A Coming Attraction

Git has a feature called **worktrees**: multiple working directories sharing the same repository, so you can have feature-A checked out in one folder and feature-B in another, simultaneously, no stashing.

Claude Code leans into this hard. You can have one Claude session refactoring the AKI pipeline in `aidi-extractions-feature-aki/`, another writing tests in `aidi-extractions-tests/`, and a third reviewing a teammate's PR in `aidi-extractions-review/` — all backed by the same git repo, none of them stepping on each other.

We give worktrees their own chapter later (Chapter 17). For now, just know they exist, and that they're the right answer when you find yourself wanting to "pause this and try something else without losing my place."

The Two Rules About Force and Amend

There are two git operations Claude Code will *not* do unless you explicitly ask:

- `git commit --amend`: rewriting the previous commit. If the previous commit was pushed and shared, amend rewrites history and breaks teammates' clones.
- `git push --force` (and `--force-with-lease`): rewriting the remote branch's history. Same problem, bigger blast radius.

Claude will warn you if you ask for either. On a shared branch (especially `main`), it will push back. You can override, but you have to mean it.

 **Warning.** `git push --force` to `main` has ended careers. Don't ask Claude to do it. If you must rewrite shared history, do it yourself, slowly, with the team watching.

Similarly:

- `--no-verify` (skipping pre-commit hooks): Claude won't do it unless you ask. Pre-commit hooks exist for a reason. If a hook is failing, fix the underlying problem, don't bypass.

A KHCC Walkthrough: A Clinician's Git Day on aidi-extractions

You start the morning with a bug report: the chemo prep extractor is missing the protocol name for cyclophosphamide-based regimens. You open Claude Code.

```
/clear
```

```
Open @notebooks/chemo_prep_extractor.py. There's a bug:
cyclophosphamide protocols are returning null for protocol_name.
Find the cause, fix it, and explain what you changed.
```

Claude reads the file, finds the regex that misses "cyclo-" prefixed names, proposes a fix, and shows the diff. You read the diff. It's clean: one line of regex, one line in the test case. You approve.

```
Stage the change. Show me the proposed commit message.
```

Claude proposes:

```
fix(chemo): handle cyclo-prefixed protocol names

The protocol-name extractor regex required a word boundary
that excluded hyphen-prefixed variants like "cyclo-CHOP".
Loosened the boundary and added a unit test for the variant.
```

You approve. Claude commits.

```
Run the test suite.
```

Claude runs `pytest tests/test_chemo_prep.py`, all green.

```
Push the branch and open a draft PR against main. Title
under 70 chars. Include a test plan with three boxes:
unit tests pass, eval cohort re-run, downstream alert
volume check.
```

Claude pushes, runs `gh pr create --draft`, prints the URL. You click. You review. You toggle the PR to "ready for review." You go get coffee.

Total time: about eight minutes. Time you spent on git plumbing: zero. Time you spent reading the diff and judging the change: about three minutes. That's the right ratio.

 **Remember.** You are the reviewer. Claude is the typist. Don't reverse those roles.

Try This

1. In any repo, make a small change (a typo fix in a comment, even). Ask Claude to stage, write a commit message, and show you the message before committing. Read the message. Edit if needed. Commit.
2. Open a draft PR with Claude. Read the body Claude wrote and compare to what you would have written. Note which is more useful.
3. Run `gh pr list` through Claude and ask for a summary of open PRs in a repo you care about. Notice how clean the output is compared to clicking through GitHub's UI.
4. Practice the discipline: every time Claude proposes a commit, *read the diff first*. Build the muscle.

Watch Out

- **Don't trust commit messages without reading the diff.** Claude writes good messages, but the message reflects the diff, not the intent. If the diff is wrong, the message is confidently wrong.
- **Don't `--force` push, ever, on a shared branch.** If Claude proposes it, push back.
- **Don't `--no-verify` to skip a failing hook.** Fix the hook's complaint.
- **Don't conflate checkpoints with commits.** Checkpoints rescue you mid-session. Commits are the durable record. You need both.
- **Don't let Claude do git on a repo you haven't backed up.** First-time use, on a non-toy repo: have a remote, push often. Git is your safety net only if it actually exists.

Chapter 9: Plan Mode and Thinking

Sometimes you don't want Claude to start typing. You want it to *stop and think*, and the single keystroke that changes the outcome of your week is Shift+Tab.

Plan Mode: What It Is

Press **Shift+Tab** in Claude Code. The status line changes. You're now in **Plan Mode**.

In Plan Mode, Claude can do everything *read-only*:

- Read files.
- Glob and Grep the repo.
- Run read-only Bash (`ls` , `git status` , `git log` , `wc -l` , you get the idea).
- Look at screenshots you paste.
- Think hard.

Claude **cannot** do anything that changes the world:

- No Write.
- No Edit.
- No `git commit` , no `pip install` , no `rm` .
- No destructive Bash.

What Claude produces in Plan Mode is *a plan*: a written breakdown of what it would do, file by file, change by change, with the rationale. You read the plan. You approve it (or you don't). Only after you approve does Claude exit Plan Mode and execute.

 **Remember.** Plan Mode is read-only by design. It's the safe room you walk Claude into before you let it touch anything important.

When to Use Plan Mode

Use it when *the cost of doing the wrong thing is higher than the cost of slowing down*.

Concretely:

- **Multi-file changes.** Any refactor that touches more than two files. The blast radius of "Claude misunderstood" multiplies with file count.

- **Structural changes.** Renaming a function used everywhere. Changing a schema. Splitting a module.
- **Anything you'd waste 20 minutes on.** If you'd burn an afternoon undoing the wrong version, plan first.
- **Anything you don't fully understand yourself.** "I want to add evals to this pipeline, I'm not sure where the prompts live or how they're loaded." Make Claude read first, plan second, execute third.
- **Production-touching changes.** Anything that's going to run against patient data, anything an alert depends on, anything in `main`.

When *not* to use it:

- One-line fixes you can see in your head.
- Typo fixes.
- Renaming a single variable in a single file.
- Generating boilerplate that's obvious.
- Exploring. "What does this codebase look like?" Plan Mode is overkill for browsing.

 **Tip.** A good rule of thumb: if the prompt starts with "refactor", "restructure", "migrate", or "add evals to", you want Plan Mode. If it starts with "fix typo" or "rename `x` to `y`", you don't.

What a Good Plan Looks Like

A plan from Claude in Plan Mode should answer four questions:

1. **What files will change, and how.** Each file gets a line. Each change gets a one-sentence description.
2. **What new files will be created.** With purpose.
3. **What I'm assuming.** Listed explicitly. If those assumptions are wrong, you catch them now, not after the changes are live.
4. **What I'm leaving for the human.** Anything Claude can't decide (a credential, a clinical threshold, a naming choice).

If you get back a plan that's three vague bullet points, the prompt was vague. Ask again with more specificity.

 **Remember.** A plan you can't critique is a plan that isn't ready. If you can't find anything to push back on, either the change is genuinely trivial or you didn't read carefully.

Ctrl+G: Edit the Plan Before You Execute

This is the underused half of Plan Mode.

Once Claude has produced a plan, you can hit **Ctrl+G** to open the plan in your `$EDITOR` (vim, nvim, VS Code, whatever you've configured). You can:

- Delete steps you don't want.
- Add steps Claude missed.
- Rewrite the rationale for steps that were vague.
- Reorder, comment, tighten.

When you save and close the editor, the *edited* plan is what Claude executes from.

 **Tip.** Ctrl+G is where the senior engineer hides. You can write Claude's marching orders precisely, in your own language, with the surgical detail you'd want from a thoughtful human collaborator. Use it on anything non-trivial.

This is the difference between "Claude plans, you approve" and "Claude drafts, you direct." The second one ships better software.

Extended Thinking: What's Happening Underneath

Behind Plan Mode (and increasingly behind regular prompts) there's **extended thinking**: a dedicated reasoning budget the model spends *before* it writes the visible response.

On Opus 4.6 and Sonnet 4.6, extended thinking is on by default with an adaptive budget. The model decides how much to think based on the difficulty of the prompt. You usually don't see the thinking; you see the output that came from it.

You can nudge the budget with the legacy keywords:

- `think` : modest extra budget.
- `think hard` : more.
- `think harder` : more still.
- `ultrathink` : the cap.

These are not magic incantations. They're literal hints to the inference engine to allocate more reasoning tokens. On Opus 4.6, `ultrathink` will visibly slow the response down by 30 seconds to a couple of minutes, and on the right problem, it'll be the difference between a working solution and a near-miss.

 **Technical Stuff.** Thinking tokens are billed like output tokens. `ultrathink` on Opus can spend tens of thousands of tokens before the model writes a word of visible response. Use it when the problem deserves it. Don't sprinkle it on every prompt.

Plan Mode Plus Ultrathink: The Heavy Combo

For a genuinely gnarly task (say, refactoring the AKI staging logic so the pipeline can swap between KDIGO 2012 and KDIGO 2024 without rewriting the alert layer) you want both:

1. Shift+Tab into Plan Mode.
2. Switch to `/model opus` if you aren't already.
3. Prompt with `ultrathink`.

`ultrathink`

Read the AKI staging code in `@notebooks/aki_extraction.py` and the alert layer in `@notebooks/aki_alerts.py`. We need to add support for KDIGO 2024 staging while keeping KDIGO 2012 as the default. The pipeline runs nightly against live patient data. Plan a refactor that:

- Keeps the existing 2012 behavior identical (the eval suite must pass unchanged).
- Introduces a config flag (default 2012) for staging version.
- Isolates the staging logic so future versions are drop-in.
- Does NOT touch the alert layer's API.

Surface any assumption you're making. Surface anything you're not sure about. Don't write code yet.

What you get back is a plan that's been *thought through*, not just sketched. You critique it. You Ctrl+G it. You execute when you're ready.

 **Tip.** The cost of running Opus + `ultrathink` + Plan Mode on a hard task is maybe a dollar or two. The cost of executing the wrong refactor and reverting it is a half-day. Math is easy.

When Planning Is Overkill

Two real examples, to calibrate.

Overkill: "Add a docstring to this function." You don't need a plan. You need three lines of code. Just type the prompt.

Overkill: "Rename `compute_baseline` to `compute_baseline_creatinine` in this one file." A grep-and-replace. No planning needed.

Not overkill: "Rename `compute_baseline` everywhere it's used in the repo and update all callers, tests, and docstrings." This touches many files. There may be string occurrences that look like the function name but aren't. Plan Mode catches all of it before you make the change.

Not overkill: "Add evals to the chemo prep extractor." This is going to touch the prompt module, add a new test file, modify CI, and probably create a new table in `aidi_catalog`. You want the plan before you start typing.

 **Remember.** The bigger the blast radius, the more Plan Mode earns its keep. Trivial changes don't need it. Anything you'd regret getting wrong does.

A KHCC Walkthrough: Refactoring AKI Staging in PySpark

You're sitting down to refactor the AKI staging logic. The current code computes KDIGO 2012 stages inline inside the main extraction notebook, mixed with the alerting logic. You want to pull it into a separate, testable module.

Step 1. Shift+Tab into Plan Mode. `/model opus`.

Step 2. Prompt:

```
ultrathink

Read @notebooks/aki_extraction.py and @notebooks/aki_alerts.py.

Goal: extract the KDIGO staging logic into a new module
notebooks/staging/kdigo.py with two functions:
- stage_kdigo_2012(creatinine_series, baseline) -> stage
- stage_kdigo_2024(creatinine_series, baseline) -> stage

The main extraction notebook should import from there.
The alert layer should not change.

Plan the refactor. Identify:
- Every place the 2012 staging is currently computed inline.
- Every place baseline_creatinine is computed (so we know
  what inputs to expect).
- The eval cases that protect existing behavior.
- Any place this might break (silent type coercion,
  pandas vs PySpark series, null handling).

Do not write code. Plan only.
```

Step 3. Claude produces a 9-step plan, listing 4 files to change, 1 new file to create, 3 assumptions, and 2 open questions ("should we keep the inline function as a deprecation shim or remove immediately?").

Step 4. You hit Ctrl+G. You edit. You answer the open questions in the plan itself, you add a step Claude forgot ("add a unit test for the boundary case where baseline is null"), you remove a step Claude added that's out of scope.

Step 5. You save. You exit Plan Mode (Shift+Tab again). You tell Claude: "Execute the plan."

Step 6. Claude works through the steps. You watch the diffs. You run the eval suite at the end. It passes unchanged.

Total time: maybe 90 minutes for a refactor that would have been a half-day flailing without a plan.

 **Remember.** The eval suite is the verification gate. Before-and-after the refactor, the deceased-patient eval cohort must produce identical alert counts. If it doesn't, you didn't refactor — you regressed.

Try This

1. In any repo, pick a small refactor task. Shift+Tab into Plan Mode. Read the plan. Even if you don't execute, notice how the plan differs from what you'd have done by just typing.
2. On a harder task, run Plan Mode with `ultrathink` on Opus. Compare the plan to a Plan Mode run on Sonnet without `ultrathink`. Feel the difference.
3. The next time you get a plan back, hit Ctrl+G and edit it before approving. Add a step Claude missed. Remove one you don't want. See how the executed change tracks your edits exactly.
4. Try Plan Mode on a task you *think* is trivial. If the plan is one line, you were right; just type the prompt next time. If the plan shows three files you forgot about, you were wrong.

Watch Out

- **Don't approve a plan you didn't read.** That's just adding ceremony without adding safety.
- **Don't use `ultrathink` on every prompt.** It's expensive and most prompts don't deserve it. Save it for the genuinely hard ones.
- **Don't skip Plan Mode on multi-file refactors because you're "in a hurry."** The hurry is what produces the broken refactor. Plan, execute, ship.
- **Don't exit Plan Mode and forget to come back.** It's per-session and per-toggle. If your next task also deserves planning, toggle it again.

- **Don't confuse "Claude wrote a plan" with "Claude executed the plan."** Plan Mode is read-only. Until you exit it and tell Claude to execute, nothing has changed on disk. That's a feature, not a bug.

PART III

Making Claude Code Yours

Chapter 10: CLAUDE.md, Your Project Instruction Manual

Every project has a culture, and Claude shows up to work on day one knowing nothing about yours.

`CLAUDE.md` is the onboarding document you hand it before the first commit.

`CLAUDE.md` is a plain markdown file that lives in the root of your project. Claude Code reads it before every single session, before the first prompt, before it looks at a single line of code. Whatever you put in there becomes the operating manual that frames every decision Claude makes for the rest of the conversation. Conventions, preferences, the names of the tables it's allowed to touch, the encoding scheme for MRNs, the eval suite it must run before declaring victory: all of it goes here.

If you ignore `CLAUDE.md`, Claude will guess. And in a clinical AI codebase, guessing is how a patient ends up missing from an AKI alert run.

What CLAUDE.md Actually Does

When Claude Code starts up in a project directory, it walks the file tree looking for `CLAUDE.md` files at three levels:

1. **Personal:** `~/.claude/CLAUDE.md`. Your private global rules. Applies to every project you open. Things like "I use uv, not pip" or "always quote PowerShell paths with spaces."
2. **Project:** `./CLAUDE.md` at the repo root. The team's shared rules. Checked into git so everyone gets the same context.
3. **Directory:** a `CLAUDE.md` anywhere deeper in the tree. Applies only when Claude is working in that subdirectory. Useful for module-specific conventions (e.g., `notebooks/extraction/CLAUDE.md` listing the AIDI extraction patterns).

All three get concatenated into the system context. They cost tokens. They drift quality if they're too long. They are the single most important configuration step in Claude Code.

 **Remember.** `CLAUDE.md` is read every session, before every turn. It is not a README. It is not for humans. It is a system prompt addendum, and every line of it spends attention from your context window.

Step One: `/init`, Then Cut It in Half

The fastest way to get a `CLAUDE.md` is to run `/init` inside Claude Code. It scans your repo, infers your stack, and writes a starter file: framework, package manager, build commands, test commands, directory layout.

Then (and this is Suryansh Tiwari's rule #27) you cut the result in half. Maybe more. `/init` is generous. It produces a 200-line file when 40 lines would do. Read every line and ask: did Claude actually need this? If your repo is obviously a Django app, the line "this is a Django app" is a wasted token.

The litmus test (Suryansh #28): for every line, ask **"would Claude make a mistake without this line?"** If the answer is no, delete it.

 **Tip.** Run `/init` in a brand new project, immediately commit the generated file to git, then cut it in half on a second commit. The diff between the two commits is your record of what mattered.

The Instruction Budget

There's a soft ceiling. Anthropic's own teams and the community have converged on roughly **150–200 instructions** before compliance starts dropping. Beyond that, Claude follows your rules less reliably. Not because the model can't see them, but because attention gets diluted across a wall of prose.

So treat `CLAUDE.md` like a budget. Every rule you add subtracts from the attention available for the next one. If you find yourself writing the 47th bullet about Python imports, ask whether a hook (Chapter 14) wouldn't do the job for free, with zero context cost.

 **Warning.** A bloated `CLAUDE.md` doesn't just cost tokens. It quietly degrades adherence to the rules you actually need. More is not better. The rule you needed Claude to follow is now competing with 40 lines about your preferred indentation.

Auto-Update After Mistakes

Suryansh's rule #29 is the one most people skip and the one with the highest compounding return. Every time Claude makes a mistake (picks the wrong table, forgets to encode an MRN, uses `pip` when you told it `uv`) say:

"Update `CLAUDE.md` so this doesn't happen again."

Claude will append a line. Over the course of a project, your `CLAUDE.md` becomes a living record of every lesson learned. The bugs you've already paid for become bugs you won't pay for twice.

 **Remember.** Your `CLAUDE.md` should be growing as your project matures. If it hasn't changed in a month, you're either not catching mistakes or not feeding them back into the file.

`.claude/rules/` for Conditional Rules

Some rules only apply in some folders. TypeScript rules don't help when Claude is working on R scripts. Azure SQL rules don't help in a frontend directory. Stuffing all of them into the global `CLAUDE.md` wastes context every time Claude is working somewhere they don't apply.

The fix is `.claude/rules/` (Suryansh #30). Each file there can declare a path frontmatter:

```
---
paths:
  - "notebooks/extraction/**/*.py"
  - "notebooks/extraction/**/*.ipynb"
---

# AIDI Extraction Rules

- Always use PydanticAI for structured LLM extraction.
- Default model is `gpt-4.1-mini`.
- Decrypt names with Fernet, encode MRNs with Optimus.
- Never log MRNs in plaintext.
```

Claude only loads that file when working on a path that matches. Zero context cost when you're elsewhere.

`@imports` to Keep the Main File Lean

You can reference another file from inside `CLAUDE.md` with `@`:

```
For git conventions, see @docs/git-conventions.md.
For the deceased-patient eval workflow, see @docs/eval-cohort.md.
```

Claude loads the referenced file only when the topic actually comes up. Your base `CLAUDE.md` stays small; the long-form policies live in their own files where humans can read them too (Suryansh #31).

 **Technical Stuff.** `@imports` are resolved relative to the location of the `CLAUDE.md` that contains them. They can be nested — an imported file can `@import` further files. Don't go more than two levels deep; debugging gets hard.

The Auto-Memory File

There's also a memory file Claude maintains on its own. Tell it "remember that I always use tidymodels for R ML work" and it will write that fact into a memory file Claude reads on subsequent sessions. You don't manage it directly, but you can ask Claude to show you what's in there, add to it, or remove things.

Think of it as: `CLAUDE.md` is your hand-written manual; the memory file is the running notebook Claude keeps about you.

A Real KHCC `CLAUDE.md`

Here's a starter for an AIDI clinical AI repository. Cut it down to fit your project. The goal is the shortest file that prevents the mistakes you've actually seen.

```
# KHCC AIDI Pipeline – CLAUDE.md

## Stack
- Databricks notebooks (PySpark + Python cells).
- Azure OpenAI `gpt-4.1-mini` is the default model. PydanticAI for structured extraction.
- R: tidymodels, finalfit, gtsummary, survminer (Lancet theming).
- Unity Catalog: `aidi_catalog`. Azure SQL: `AIDI-DB`.

## Patient data – non-negotiable
- Never log MRNs or names in plaintext.
- MRNs: Optimus-encode before any output (email, dashboard, log).
- Names: Fernet-encrypt before storage.
- Eval cohorts use the frozen deceased-patient set in `aidi_catalog.dbo.eval_cohort`. Never :

## Eval gate
- No prompt or extraction change ships without an eval run on the deceased-patient cohort.
- Results table: `aidi_catalog.dbo.eval_runs`. Append-only. Never overwrite.

## The 12 rules (Karpathy + Mnilax)
1. Think before coding. Surface assumptions; ask rather than guess.
2. Simplicity first. Minimum code that solves the problem.
3. Surgical changes. Touch only what you must. Don't "improve" adjacent code.
4. Goal-driven execution. Define success, then loop until verified.
5. Use the model only for judgment calls. KDIGO staging is rules – compute it. Don't ask the
6. Token budgets are not advisory. 4k per task, 30k per session. Surface breaches.
7. Surface conflicts, don't average them. Pick one pattern; flag the other.
8. Read before you write. Read the file, the immediate caller, and shared utilities.
9. Tests verify intent, not behavior. Eval cases must test the clinical intent.
10. Checkpoint after every significant step.
11. Match the codebase's conventions, even if you disagree. Raise objections separately.
12. Fail loud. Silent failures are the most dangerous failure mode.

## Conventions
- See @docs/aidi-extractions.md for the extraction pipeline pattern.
- See @docs/email-pattern.md for the `DoNotReply@khcc.jo` sender pattern.
- Commit messages: `type(scope): subject`. Scope = pipeline name.

## Communication
- Concise. No preambles. Lead with the diff.
- Push back honestly. Diplomatic hedging wastes time.
```

That's about 50 lines of actual instructions. Under budget. Every line passes the litmus test.

 **Tip.** Notice the use of `@docs/...` imports. The 12 rules and patient-data section live in the main file because they apply to every interaction. The pipeline-specific patterns live in imported files that load only when relevant.

Try This

1. In a project of yours that doesn't have a `CLAUDE.md`, run `/init`. Read every line of the generated file out loud.
2. Delete every line you can't justify with the litmus test ("would Claude make a mistake without this?").
3. Now make Claude do something. When it makes its first mistake (it will), say "update `CLAUDE.md` so this doesn't happen again." Read what it adds.
4. Move one section out into a separate file under `docs/` and replace it with an `@import`. Notice that nothing breaks.

Watch Out

- **Don't paste your README into `CLAUDE.md`.** READMEs are for humans. `CLAUDE.md` is a system prompt. Different audience, different rules.
- **Don't put secrets in `CLAUDE.md`.** It's checked into git. Treat it as public.
- **Don't write rules that contradict each other.** Claude will follow the first one it reads and quietly violate the second. If two rules conflict, pick one and delete the other.
- **Don't forget the personal layer.** Your `~/ .claude/CLAUDE.md` carries across every project. If you stuff project-specific rules there, they'll leak into projects where they don't apply.
- **Don't let it grow forever.** Every quarter, re-run the litmus test on every line. Delete what's no longer earning its place.

Chapter 11: Slash Commands

A slash command is a keystroke that replaces a paragraph: the difference between typing "please clear the conversation history and start fresh" and typing `/clear`.

Slash commands are the keyboard shortcuts of Claude Code. Some are built in. Some you write yourself. All of them collapse a recurring instruction into a single token and a tab key. After a week of using Claude Code seriously, your fingers will know `/clear`, `/cost`, and `/compact` the way they know Ctrl+C.

The Built-Ins Worth Knowing

There are dozens of built-in slash commands. You don't need to memorize them all. `/help` lists them at any time. But these are the ones that matter for daily work.

Session and conversation control:

- `/init`. Scan the project and write a starter `CLAUDE.md`. Run this once at the start of any new project. Then cut the result in half (see Chapter 10).
- `/clear`. Wipe the conversation. New session, empty context. Use this between unrelated tasks. A clean session beats a messy three-hour one.
- `/compact`. Summarize the conversation so far and continue with a smaller context. Use when you're deep in a session but need to keep going. You can pass guidance: `/compact focus on the SQL changes` keeps the parts you care about.
- `/rewind`. Jump back to a previous checkpoint. Claude saves a snapshot before every edit; `/rewind` opens a list and lets you restore either the code, the conversation, or both. This is your time machine.

Cost and model:

- `/cost`. How much have you spent in this session?
- `/stats`. Detailed token breakdown. Input tokens, output tokens, cached tokens, per-model.
- `/model`. Switch models mid-conversation. `/model haiku` for cheap tasks, `/model sonnet` for normal work, `/model opus` for the hard ones. Sonnet 4.6 and Opus 4.6 support `/model opus[1m]` for a million-token context window.

Configuration:

- `/config`. Open the config UI. Pick your output style (Explanatory, Concise, Technical) once and forget about it.

- `/permissions` . Manage what Claude can do without asking. Allowlist the safe stuff (`npm test` , `git status`); keep the risky stuff (`rm` , `DROP TABLE`) gated.
- `/statusline` . Install a live HUD at the bottom of your terminal showing directory, branch, and context usage. Once you have it, you'll wonder how you worked without it.
- `/help` . The list. When you forget, type this.

Productivity:

- `/btw` . Pop an overlay for a quick side question that doesn't pollute the main thread. "By the way, why did you pick `async` here?" Ask, get the answer, your main conversation is undisturbed.
- `/voice` . Push-to-talk dictation. Counterintuitively, speaking a prompt usually produces better prompts than typing one; you naturally add more context and constraints.
- `/sandbox` . Run Claude with OS-level isolation (Seatbelt on macOS, bubblewrap on Linux). Let it run wild on an experimental refactor with zero risk to your actual system. Review the diff, merge what you like.
- `/loop 5m /check-eval-runs` . Schedule a slash command to run every five minutes in the background while your main session continues. Good for polling deploys, eval runs, or long jobs.
- `/agents` . List and manage subagents (Chapter 13).

 **Remember.** The two most important built-ins are `/clear` and `/compact` . They are the difference between a productive day and a session that slowly rots into confused outputs. Use them aggressively.

 **Tip.** `/rewind` is criminally underused. If Claude goes off the rails, don't argue with it for ten minutes. Hit `/rewind` , pick the prompt right before things went sideways, and try a different angle. Your future self will thank you.

Custom Slash Commands

Here's where things get interesting. Any markdown file you drop into `.claude/commands/` becomes a slash command. The filename (without `.md`) is the command name. The contents are the prompt.

That's it. No configuration, no plugin system. A file is a command.

A minimal example. Create `.claude/commands/commit.md` :


```

---
allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*), Bash(git diff:*)
argument-hint: [message]
description: Create a git commit with context
---

## Context

- Current git status: !`git status`
- Current git diff: !`git diff HEAD`
- Current branch: !`git branch --show-current`
- Recent commits: !`git log --oneline -10`

## Your task

Based on the above changes, create a single git commit.

If a message was provided via arguments, use it: $ARGUMENTS

Otherwise, analyze the changes and create a conventional commit message:
- `feat:` for new features
- `fix:` for bug fixes
- `refactor:` for code refactoring
- `test:` for adding tests
- `docs:` for documentation changes

```

Three pieces are worth noticing.

The frontmatter. `allowed-tools` pre-approves specific bash commands so Claude doesn't have to ask permission. `argument-hint` tells the user what to pass. `description` is what appears in the slash-command picker.

The `!` prefix. Lines starting with `!` are shell commands that run *before* Claude sees the prompt. Their output gets injected into the prompt itself. This is how you give Claude the current git diff without asking for it.

`$ARGUMENTS`. Whatever the user types after the command name gets substituted in. `/commit fix the AKI null handling` puts that string into the prompt.

 **Technical Stuff.** The `!` prefix and `$ARGUMENTS` substitution happen on the client side, before the prompt is sent to the model. They're cheap. They don't burn any extra tokens beyond the shell output itself.

Two More Examples Worth Stealing

Here's a `/optimize` command:

```

---
description: Analyze code for performance issues and suggest optimizations
---

# Code Optimization

Review the file(s) I just shared for the following, in priority order:

1. Performance bottlenecks:  $O(n^2)$  operations, inefficient loops, repeated work.
2. Memory issues: unreleased resources, large intermediate copies.
3. Algorithm improvements: better data structures, vectorization opportunities.
4. Caching opportunities: repeated computations that could be memoized.
5. Concurrency: race conditions, missing parallelism.

For each finding, output:
- Severity (Critical / High / Medium / Low)
- Location (file:line)
- Explanation of the problem
- Recommended fix, with a code example

```

And a `/doc-refactor` that documents while it cleans up:

```

---
description: Refactor the current file and update its docstrings
---

Refactor the file I have open. Follow these rules:

1. Extract any function longer than 30 lines.
2. Rename any variable named `x`, `data`, `tmp`, or `result` to something specific.
3. Add or update a one-line docstring on every function.
4. Do NOT change behavior. Run the test suite after you're done.
5. Show me the diff before committing anything.

```

Notice: these are just prompts. There's no DSL, no plugin manifest, no build step. A file is a command.

 **Tip.** Start your custom commands by copying ones that already exist. The `luongnv89/claude-howto` repo has a clean set under `01-slash-commands/`: `commit`, `pr`, `optimize`, `doc-refactor`, `unit-test-expand`, `generate-api-docs`. Copy the one closest to your need, tweak the prompt, ship it.

A KHCC Custom Command: `/run-aidi-eval`

Here's one that matters for AIDI work. The deceased-patient eval cohort is the gate for any prompt change. You want it to be one keystroke.

Create `.claude/commands/run-aidi-eval.md` :

```

---
allowed-tools: Bash(databricks jobs run-now:*), Bash(databricks jobs get-run:*)
argument-hint: [pipeline-name]
description: Run the deceased-patient eval suite against a pipeline and report results
---

## Context

- Pipeline under test: $ARGUMENTS
- Eval cohort: `aidi_catalog.dbo.eval_cohort` (frozen deceased-patient set)
- Results table: `aidi_catalog.dbo.eval_runs`

## Your task

1. Trigger the Databricks eval job for the pipeline `$ARGUMENTS`. Use the job ID mapped in `docs/eval-jobs.json`.
2. Poll the run until it completes (status `TERMINATED`).
3. Query `aidi_catalog.dbo.eval_runs` for the latest run of `$ARGUMENTS` and compare against the previous run on the same pipeline:
   - Accuracy delta
   - Precision/recall delta
   - Any new failure cases (rows where `prev_pass = true AND new_pass = false`)
4. Report:
   - Pass/fail summary
   - Regressions (must be zero to ship)
   - Any new wins
5. Do NOT modify any prompt or pipeline code. This command is read-only.

## Acceptance

If there are any regressions, the output must start with `EVAL FAILED`.
If zero regressions, start with `EVAL PASSED`.

```

Now anyone on the team types `/run-aidi-eval pathology-v6` and gets a clean pass/fail with a diff against the previous run. The command captures the entire convention (which table, which cohort, which acceptance gate) so nobody has to remember it.

 **Warning.** Custom commands that take bash actions are powerful and dangerous in equal measure. Always set `allowed-tools` to the narrowest set you can. A command that allowlists `Bash(*)` is a command that can `rm -rf` your repo when Claude takes a creative interpretation.

Where Commands Live

- `.claude/commands/foo.md` : project-level, checked into git, shared with the team.
- `~/.claude/commands/foo.md` : personal, available in every project.

A project-level command shadows a personal one of the same name. Keep team-relevant workflows in the repo; keep your personal quirks in your home directory.

Try This

1. Run `/help` and read every built-in command. Pick three you didn't know about.
2. Run `/cost` after your next hour of work. Note the number. Run `/clear`. Notice what changes on the next prompt.
3. Create `.claude/commands/standup.md` that prints `git log --since=yesterday --oneline` and asks Claude to summarize what you did. Run it the next morning.
4. Steal the `commit.md` example above. Adapt it for your project's commit style. Use `/commit` for the next week instead of typing commit messages by hand.

Watch Out

- **Don't overload `allowed-tools`.** `Bash(*)` is not a permission, it's a security hole. Be specific: `Bash(git status:*)`, `Bash(npm test:*)`.
- **Don't put secrets in ! shell commands.** Anything you `cat` into a prompt becomes part of the conversation and may be logged.
- **Don't make a slash command for a one-off task.** If you're going to use it twice, write one. If once, just type the prompt.
- **Don't forget to commit `.claude/commands/`.** The whole point of project-level commands is that the team gets them. If they live only on your laptop, you've reinvented a personal shortcut.
- **Don't name a custom command the same as a built-in.** It either won't override or will, and either outcome is confusing.

Chapter 12: Skills

A skill is what happens when you stop explaining the same thing to Claude every week and instead write it down once, in a file Claude reads on its own when the topic comes up.

Slash commands (Chapter 11) are *explicit*: you type `/optimize` to invoke one. Skills are *implicit*: Claude notices that the current task matches a skill's description and pulls it in automatically. You don't have to remember they exist. That's the point.

What a Skill Actually Is

A skill lives at `.claude/skills/<skill-name>/SKILL.md`. The minimum is one file with YAML frontmatter:

```
---
name: code-refactor
description: Systematic code refactoring based on Martin Fowler's methodology. Use when the
---

# Code Refactoring Skill

A systematic approach to refactoring code...
[rest of the playbook]
```

The frontmatter has two required fields: `name` and `description`. The description is the most important line in the whole file because that's what Claude reads to decide whether the skill is relevant to the current task. Vague descriptions trigger never; precise descriptions trigger exactly when they should.

The body of `SKILL.md` is the playbook itself. It can be as long as it needs to be. Claude only loads the body when it actually invokes the skill, so length here is cheap.

 **Remember.** A skill is just a folder with a `SKILL.md` in it. There's no plugin manifest, no install step, no version checking. If the file is in `.claude/skills/`, it's available. If not, it isn't.

Skills vs. Slash Commands

This trips people up at first. They look similar (both are markdown files with prompts) but they're invoked differently:

- A **slash command** is explicit. You type `/commit` and the command runs. Claude doesn't decide; you do.
- A **skill** is description-driven. When you ask Claude to do something, it scans the descriptions of all available skills and invokes the matching one on its own. You don't type a magic word; you just describe the task in plain English.

Rule of thumb:

- Use a **slash command** when *you* know you want the workflow ("commit this," "run the eval suite").
- Use a **skill** when *Claude* needs to recognize it ("the user is asking to refactor: go read the refactoring playbook").

 **Tip.** If a workflow's trigger is a verb ("refactor," "review," "draft"), it's probably a skill. If it's a noun-y shortcut ("commit," "deploy," "report"), it's probably a slash command. Some workflows want both: a skill that gets pulled in automatically and a slash command that forces it.

The Context Trick

Skills are designed to *not* inflate your baseline context. Here's how that works:

1. At session start, Claude only loads the names and descriptions of available skills. That's typically 100 characters per skill.
2. When you ask something that matches a description, Claude loads that skill's `SKILL.md` body into context.
3. When `SKILL.md` references other files (`templates/foo.md` , `references/bar.md` , `scripts/baz.py`), Claude reads those *only when it actually needs them*.

So you can have 30 skills installed and the baseline cost is 30 short descriptions. The big playbooks only get loaded when relevant.

This is the difference between a skill and just pasting the same playbook into `CLAUDE.md` . The playbook in `CLAUDE.md` costs tokens every single turn forever. The skill costs nothing until it fires.

 **Technical Stuff.** The matching is done by the model itself, not a regex. That's why the description has to be written for the model, not for you. "Use when..." and "Trigger for..." phrasings work because they're imperative instructions the model can act on.

The Anatomy of a Real Skill

Let's walk through the `code-refactor` skill end to end (from the `luongnv89/claude-howto` repo).

The folder layout:

```
.claude/skills/code-refactor/
├── SKILL.md
├── references/
│   ├── code-smells.md
│   └── refactoring-catalog.md
├── templates/
│   └── refactoring-plan.md
└── scripts/
    ├── analyze-complexity.py
    └── detect-smells.py
```

`SKILL.md` is the entry point. It defines six phases (research, test coverage, code-smell ID, plan, incremental implementation, review) and references the supporting files where the detail lives:

```
See [references/code-smells.md](references/code-smells.md) for the complete catalog.
See [templates/refactoring-plan.md](templates/refactoring-plan.md) for the planning template
```

Claude doesn't read `code-smells.md` at session start. It doesn't read it when the skill fires. It only reads it when, mid-refactor, it needs the smell catalog. The full reference is hundreds of lines, and it costs zero tokens until that moment.

Supporting `scripts/` are even better: they're shell or Python tools Claude can *execute* rather than read. `python scripts/detect-smells.py src/auth.py` runs an analyzer and returns its output. Deterministic work belongs in scripts; only the judgment calls belong in the LLM.

⚠ Warning. Skills can carry executable scripts. Anyone who installs your skill into their project is also trusting your scripts. Don't ship a skill with a `curl | bash` script in it. And don't install skills from random repos without reading the scripts first.

Three KHCC Skills Worth Building

Here are three skills that would pay rent in any AIDI workflow.

`aidi-extractions`

The description-driven trigger is what makes this powerful. Description:

"SQL Server extraction patterns, gotchas, and best practices for building clinical data cohort tables at King Hussein Cancer Center (KHCC) AIDI Department. Use this skill whenever building or modifying any SQL extraction pipeline against KHCC VISTA_ / SILVER_ / GOLD_ tables — even for simple queries. Also covers Databricks notebook coding standards, Azure OpenAI (gpt-4.1-mini) integration, PydanticAI structured extraction, email sending via Azure Communication Services, Fernet encryption, Optimus MRN encoding, and Azure TTS."

Contents:

- `SKILL.md` : the spine — connection patterns, decryption pattern, MRN encoding, the deceased-patient eval gate, the email pattern.
- `references/silver-vs-gold.md` : schema reference for the SILVER_ and GOLD_ tables.
- `references/pydanticai-patterns.md` : extraction patterns with examples.
- `templates/notebook-template.py` : a starter Databricks notebook with the cell structure, imports, and Fernet/Optimus helpers wired up.
- `scripts/verify-mrn-encoding.py` : a deterministic check that every row in an output table has Optimus-encoded MRNs.

Now any time someone says "extract chemo orders for the FN cohort," Claude pulls in the playbook without anyone having to remember it exists.

`r-clinical-analysis`

Description:

"R Markdown template and best-practice patterns for clinical/oncology research analysis. Use when writing R code for medical datasets, survival analysis, GvHD studies, BMT outcomes, or any clinical report requiring baseline characteristics tables, Kaplan-Meier curves, univariable/multivariable Cox regression, and publication-ready output. Covers data cleaning, date handling, factor conversion, gtsummary tables, finalfit Cox regression, survminer plots with Lancet theming."

Contents:

- `SKILL.md` : the analysis spine — tidyverse setup, date handling gotchas, factor conversion rules, the order of operations for survival analysis.
- `templates/clinical-analysis.Rmd` : a complete Rmd template with KHCC headers.
- `references/lancet-theming.md` : the survminer theme customizations for Lancet-style figures.
- `references/gtsummary-recipes.md` : the table-1 patterns the team uses.

scientific-writing**Description:**

"Deep research and scientific writing assistant that produces publication-ready academic documents with verified citations. Use when the user asks to write, draft, or create a scientific paper, manuscript, literature review, grant proposal, clinical report, abstract, or any academic document. Also covers IMRAD structure, CONSORT/STROBE/PRISMA reporting guidelines, and journal-specific formatting."

Contents:

- `SKILL.md` : the writing process — outline-then-prose two-stage, citation discipline, reporting-guideline compliance.
- `references/imrad-structure.md` : the IMRAD section-by-section playbook.
- `references/reporting-guidelines.md` : CONSORT, STROBE, PRISMA checklists.
- `templates/manuscript.tex` : LaTeX skeleton.
- `templates/cover-letter.md` : cover-letter template.

Notice the pattern: the `SKILL.md` is short and high-level; the heavy reference material lives in files that load on demand.

 **Tip.** Write the `SKILL.md` last. Build out the references and templates first. Then write the spine that ties them together. You'll end up with a sharper skill because you've already worked through the details.

Where Skills Live

- `.claude/skills/<name>/SKILL.md` : project-level, checked into git.
- `~/.claude/skills/<name>/SKILL.md` : personal, available across every project.

Anthropic ships a bundle of high-quality skills you can install (`docx` , `pptx` , `xlsx` , `pdf` , `scientific-writing` , and more). Browse them, copy the ones you want, adapt them. The structure is consistent.

Try This

1. Pick a workflow you explain to Claude every week. Write it down. The first draft is your `SKILL.md` .
2. Write the description twice. Once for what the skill does. Once for *when* Claude should use it. The second one is what actually triggers it; make it specific.

3. Drop the file at `.claude/skills/<your-skill-name>/SKILL.md`. Start a new session. Phrase a request that should trigger the skill. Notice Claude pulling it in.
4. Now intentionally ask something that should *not* trigger it. Notice Claude leaving it alone. If it triggers anyway, your description is too broad. Tighten it.

Watch Out

- **Vague descriptions are useless.** "Use this skill for code stuff" matches every code request and degrades into noise. Be specific about what triggers and what doesn't.
- **Skills with executable scripts are a trust boundary.** Treat them like any code you'd install: read it before running it.
- **Don't put every rule into a skill.** Universal rules belong in `CLAUDE.md`. Skills are for playbooks that are conditional on what the user is doing.
- **Don't make a skill that contradicts your `CLAUDE.md`.** When they fight, Claude blends them, and that's the worst outcome. Pick one place for each rule and stick to it.
- **Don't ship a skill without testing the trigger.** Open a fresh session, phrase three different requests that should fire it, and confirm it loads each time. Then phrase one that shouldn't, and confirm it doesn't.

Chapter 13: Subagents and Agent Teams

The cleanest way to keep your main conversation sharp is to stop doing all the work in it. Hand the noisy stuff off to a specialist with its own context window.

A subagent is a fresh instance of Claude, spawned by your main session, with its own system prompt, its own tools, and most importantly its own context. It does a task, summarizes the result, and returns. Your main session never sees the 40,000 tokens of files the subagent had to read, only the answer.

That's the whole reason subagents exist. Not parallelism. Not specialization. **Context hygiene.**

Why Subagents

Three real reasons to reach for one.

1. Context preservation. You're 90 minutes into building a feature. Your main session has the design in its head. Now you need to figure out how the payment flow currently works: that's twenty files to read. If you do that in your main session, those twenty files crowd out the design context, and Claude starts forgetting why you were building this in the first place. If you delegate to a subagent (*"use a subagent to figure out the payment flow"*) the subagent reads the twenty files, summarizes, and your main session sees only the summary.

2. Focused expertise. A clean context with a tight system prompt produces better answers than a 50-message session with the same model. A subagent with the system prompt "you are a senior code reviewer focused on security and performance" will outperform asking your main agent to switch hats mid-conversation.

3. Parallelism. You can fire off multiple subagents at once on unrelated tasks. A docs writer, a test generator, and a complexity analyzer can all run while you continue talking to the main agent.

 **Remember.** The first rule of subagents is context hygiene. If you ever find yourself thinking "this task is going to balloon my context but I only need the answer," that's a subagent.

The File Format

A subagent lives at `.claude/agents/<name>.md`. The structure mirrors a skill but with extra fields:

```

---
name: code-reviewer
description: Expert code review specialist. Use PROACTIVELY after writing or modifying code
tools: Read, Grep, Glob, Bash
model: inherit
---

# Code Reviewer Agent

You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:
1. Run `git diff` to see recent changes.
2. Focus on modified files.
3. Begin review immediately.

## Review Priorities (in order)

1. Security issues: authentication, authorization, data exposure.
2. Performance problems: O(n2) operations, memory leaks, inefficient queries.
3. Code quality: readability, naming, documentation.
4. Test coverage: missing tests, edge cases.
5. Design: SOLID principles, architecture.

## Output Format

For each issue:
- Severity (Critical / High / Medium / Low)
- Category
- Location (file:line)
- Issue description
- Suggested fix with code example

```

Three fields are worth understanding:

- **description** : the trigger. Same rules as a skill — precise descriptions trigger when they should. Including the word "PROACTIVELY" is a convention that nudges the main agent to invoke the subagent without being asked.
- **tools** : which tools the subagent can use. Often narrower than the main agent. A code reviewer shouldn't need `Write` or `Edit` . Take them away. A research subagent shouldn't need `Bash` . Take it away.
- **model** : `inherit` matches the main session's model. `haiku` runs the subagent on the cheap, fast model, perfect for grep-and-summarize tasks. `opus` only if the subagent has to reason deeply.

 **Tip.** Always set `model: haiku` for subagents whose job is read-and-summarize. Haiku is ~10x cheaper than Sonnet, and for "go find me the file that defines X" it's just as accurate.

The Hub-and-Spoke Limitation

Subagents are isolated. That's their feature, but it's also a constraint. They can only talk to the main agent. They can't talk to each other.

If you spawn a researcher, a writer, and a reviewer as three subagents:

- The researcher can't show its findings to the writer directly. The main agent has to pass them.
- The reviewer can't see the researcher's notes. The main agent has to summarize them first.
- Everything bottlenecks through the central session.

For three subagents this is fine. For ten, the main agent becomes a switchboard operator and quality degrades.

 **Technical Stuff.** This isn't a bug. It's a deliberate safety property: subagents can't coordinate behind your back, can't conspire, can't accidentally form a feedback loop. The main agent is the only interface, which means you're the only auditor that matters.

Agent Teams: When the Hub Becomes a Bottleneck

For complex builds (a full app with backend, frontend, and tests, or a multi-module refactor) the hub-and-spoke pattern breaks down. Agent teams are the answer.

Enable them with the environment flag:

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Now when you ask for an agent team, Claude spins up a *team lead* plus 3–5 teammates. Each teammate runs in its own Claude Code instance. They share a task list. They can read each other's outputs from that shared list. The team lead orchestrates.

It's a different category of work. You're not delegating one task; you're delegating an entire project module.

A simple decision tree:

- **One feature, one file, one head?** → Single agent.

- **Need a clean context window or a focused specialist?** → Subagent.
- **Multi-module project where teammates need to see each other's work?** → Agent team.

⚠ Warning. Agent teams are experimental. They burn tokens fast: multiple parallel instances, each maintaining its own context. Don't enable agent teams for a one-day project. The overhead isn't worth it unless the work genuinely spans days and modules.

Three Subagents Worth Stealing

A Haiku-based quick-search subagent

The classic "go find me X without polluting my context." Description: "Searches the codebase for symbols, definitions, or usage patterns. Use proactively when the user asks where something is defined or how it's used."

```
---
name: quick-search
description: Searches the codebase for symbols, definitions, or usage patterns. Use proactively
tools: Read, Grep, Glob
model: haiku
---
```

You are a code search specialist. When invoked:

1. Identify the symbol, file, or pattern the user is looking for.
2. Use Glob and Grep aggressively to locate it.
3. Read only the files that contain matches.
4. Return:
 - A one-line summary
 - File:line locations of every relevant match
 - A 5-line context snippet for the most important match

Be concise. Do not paste large file contents. The main agent does not need them.

Haiku is fast and cheap. Search is mostly reading. Perfect match.

A strict-TypeScript reviewer

Description: "Reviews TypeScript code for type-safety violations, anti-patterns, and missing strictness. Use proactively after any TypeScript edit."

```

---
name: strict-ts-reviewer
description: Reviews TypeScript code for type-safety violations, anti-patterns, and missing
tools: Read, Grep, Bash
model: inherit
---

You are a TypeScript strict-mode enforcer.

When invoked:
1. Run `git diff` to see what changed.
2. For every changed `.ts` or `.tsx` file, check for:
  - `any` types: flag every one.
  - `as` casts: flag any that lose precision.
  - Non-null assertions (`!`): flag and demand a runtime check instead.
  - Missing return types on exported functions.
  - `// @ts-ignore` or `// @ts-expect-error`: flag and demand a reason.

Report findings inline. Do not modify code; recommend fixes only.

```

A docs writer

Description: "Writes or updates documentation (READMEs, docstrings, API docs) based on recent code changes. Use when the user asks for docs or after a feature merges."

```

---
name: docs-writer
description: Writes or updates documentation (READMEs, docstrings, API docs) based on recent
tools: Read, Glob, Write, Edit, Bash
model: inherit
---

You are a documentation writer. Your job is to keep docs in sync with code.

When invoked:
1. Run `git diff main` to see what changed since the main branch.
2. Identify which files have user-visible API changes.
3. For each:
  - Check whether a README, docstring, or doc page references the API.
  - Update it if it's now stale.
  - Add a new doc entry if the API is new and undocumented.
4. Report what you changed and why.

Keep prose tight. Code comments explain why, not what.

```

A KHCC Subagent: The Deceased-Patient Eval Runner

Here's one that earns its keep. The eval suite against the deceased-patient cohort is the gate for any prompt change. Running it pulls in tens of thousands of tokens of cohort metadata, prompt files, and result tables. You don't want that in your main session.

`.claude/agents/aidi-eval-runner.md :`

```

---
name: aidi-eval-runner
description: Runs the KHCC deceased-patient eval suite against an AIDI pipeline and reports
tools: Bash, Read
model: inherit
---

You are the KHCC AIDI eval runner. Your only job is to run the deceased-patient
eval suite and report results.

## Inputs
The user will specify a pipeline name (e.g. `pathology-v6`, `er-extractor`,
`aki-notification`).

## Procedure
1. Look up the Databricks job ID for the pipeline in `docs/eval-jobs.json`.
2. Trigger the eval job with `databricks jobs run-now --job-id <id>`.
3. Poll `databricks jobs get-run` until the run completes.
4. Query `aidi_catalog.dbo.eval_runs` for:
   - The latest run for this pipeline (call it `current`).
   - The previous run for this pipeline (call it `previous`).
5. Compute the delta:
   - Accuracy, precision, recall changes.
   - New failures: rows where `previous.pass = true AND current.pass = false`.
   - New wins: rows where `previous.pass = false AND current.pass = true`.

## Output (return only this; no preamble)
- **VERDICT**: `EVAL PASSED` (zero new failures) or `EVAL FAILED` (any new failure).
- **Summary**: accuracy delta, precision delta, recall delta.
- **New failures**: list each by case ID with a one-line diff.
- **New wins**: list each by case ID.

## Hard rules
- Never modify any pipeline code, prompt, or eval table.
- Never include living-patient data in any output.
- If the job fails to start or the table query returns no rows, report
  `EVAL INCONCLUSIVE` and stop.

```

Now the main session can stay focused on prompt edits. When you're ready to verify, *"use the aidi-eval-runner subagent to evaluate pathology-v6"* spins up a clean instance, runs the gate, and reports back without ever showing your main context the eval cohort itself.

 **Tip.** A subagent's system prompt is also a contract. Spell out what it must *not* do as clearly as what it must. The `aidi-eval-runner` is explicitly told never to modify code; that one line prevents a whole class of accidents.

Where Subagents Live

- `.claude/agents/<name>.md` : project-level, checked in, shared.
- `~/.claude/agents/<name>.md` : personal, available everywhere.

Use `/agents` to list them, edit them, or spawn one explicitly.

Try This

1. Run `/agents` and see what's already installed. (If you've installed any Anthropic skill bundles, you may have a few.)
2. Write a `quick-search` subagent like the one above. Set `model: haiku`. Use it next time you need to find where a function is defined. Watch how little context lands in your main session.
3. Pick a task that's currently expensive in context: "look at all the files in `src/auth/` and summarize the authentication flow." Once with the main agent, once via a subagent. Compare `/cost` and the context bar.

Watch Out

- **A subagent doesn't see your conversation.** It only sees the prompt you (or the main agent) hand it. If you want it to know something, include it in the invocation.
- **Subagents don't share state.** They can't talk to each other. If you need coordination, you need an agent team, or you need to redesign so they don't.
- **Too many subagents is worse than none.** Spawning five subagents to coordinate a small task is overhead. Use them when context hygiene actually matters.
- **A subagent with `Write` and `Bash` is a full Claude Code session.** Restrict tools aggressively. A code reviewer should be read-only. A search subagent should never need to write files.
- **Don't run agent teams casually.** They burn tokens in parallel. Save them for genuinely complex, multi-module work.

Chapter 14: Hooks

Everything we've covered so far depends on Claude *choosing* to follow your rules. Hooks are what you reach for when *choice* is no longer good enough.

A hook is a shell command that Claude Code runs automatically at a lifecycle event: before a tool runs, after a tool runs, when a session starts, when the agent stops. The hook always fires. It costs zero tokens. It's not advice for the model. It's plumbing.

Suryansh Tiwari's rule #33 puts it perfectly: `CLAUDE.md` **for suggestions. Hooks for requirements.**

The Lifecycle Events

There are several hook types. The four that matter most for daily work:

- **PreToolUse** : Fires *before* Claude is allowed to use a tool. Can inspect the tool name and arguments. Can block the action by exiting with a non-zero status. This is your safety net.
- **PostToolUse** : Fires *after* a tool completes. Cannot block (the action already happened) but can react — format the file, run a linter, log the change, notify a webhook.
- **Stop** : Fires when the agent finishes its turn and is about to return control to you. This is the magic ingredient in the Ralph loop (Chapter 18); a `Stop` hook can re-feed Claude its own task list and keep it working overnight.
- **SessionStart** / **SessionEnd** : Fire at the bookends of a session. Good for logging, environment setup, or printing a daily reminder.

 **Remember.** Hooks always run. There is no "Claude decided not to" with a hook. If the hook is configured, the hook fires, every time, no exceptions. That's why they're the right tool for non-negotiable rules.

Where They Live

Hooks are configured in `.claude/settings.json` under the `"hooks"` key:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"${CLAUDE_FILE_PATH}\" 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

Three pieces:

- **PostToolUse** : the event.
- **matcher** : a regex against the tool name. `Edit|Write` means "any time Claude edits or writes a file." Use `Bash` to match shell commands; use `.*` to match every tool.
- **command** : the shell command to run. Environment variables (`$CLAUDE_FILE_PATH` , `$CLAUDE_TOOL_INPUT` , etc.) are populated with the context of the tool call.

 **Technical Stuff.** The exact set of environment variables depends on the tool and the hook event. `PostToolUse` on `Edit` gives you `$CLAUDE_FILE_PATH`. `PreToolUse` on `Bash` gives you `$CLAUDE_TOOL_INPUT` with the full command. Check the docs for the full table, and when in doubt, write a hook that just prints `env` and read what's available.

Example 1: Auto-Format with Prettier

Suryansh's rule #34. The hook above runs Prettier on every file Claude edits or writes. The `|| true` keeps the hook from failing (and blocking subsequent work) if Prettier doesn't recognize the file type. The `2>/dev/null` suppresses noise.

Drop that block into `.claude/settings.json` and you will never again ask Claude to "remember to format the file." It happens automatically. Forever.

Same idea for Python:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "ruff format \"${CLAUDE_FILE_PATH}\" 2>/dev/null || true; ruff check --"
          }
        ]
      }
    ]
  }
}
```

R:

```
{
  "type": "command",
  "command": "Rscript -e \"styler::style_file('${CLAUDE_FILE_PATH}')\" 2>/dev/null || true"
}
```

 **Tip.** Stack multiple `PostToolUse` hooks for different file types. The matcher is regex against the *tool name*, not the file. Filter inside the command itself with `case "${CLAUDE_FILE_PATH}" in ... esac`.

Example 2: Block Destructive Commands

Suryansh's rule #35. The other side of the safety net: a `PreToolUse` hook that inspects every bash command and refuses the dangerous ones.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "if echo \"\$CLAUDE_TOOL_INPUT\" | grep -qE '(rm\\s+-rf\\s+|DROP\\s+T"
          }
        ]
      }
    ]
  }
}
```

If the regex matches, the hook exits non-zero, and Claude Code refuses to run the command. The model sees the rejection and tries something else.

This is what makes it safe to run Claude with broader permissions. The hook is a hard floor under your trust. Even if Claude decides creatively that `rm -rf` is the right answer, the hook stops it.

⚠ Warning. A hook is only as good as its regex. The example above blocks `rm -rf /` but not `rm -rf $HOME` or `rm -rf ~`. Write your blocklist for *your* worst-case scenarios. And remember: a hook can be bypassed if Claude finds another path (e.g., `find / -delete`). Hooks are a floor, not a ceiling.

Example 3: Log Every Edit

Audit trails matter, especially in a regulated environment.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "echo \"$(date -u +%Y-%m-%dT%H:%M:%SZ) $CLAUDE_FILE_PATH\" >> .claude,"
          }
        ]
      }
    ]
  }
}
```

Now every file Claude touches is recorded with a timestamp. No token cost. No prompt engineering. Just a tail of the log when you need to know what happened.

A KHCC Hook: Protect the Eval Cohort

Here's one worth writing today. The frozen deceased-patient cohort lives in `aidi_catalog.dbo.eval_runs`. If a non-eval session accidentally writes to that table, the eval baseline is corrupted and future regressions become invisible. This is the worst-case failure mode for a clinical AI eval suite.

A `PreToolUse` hook can prevent it:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "if echo \"${CLAUDE_TOOL_INPUT}\" | grep -qE 'aidi_catalog\\.dbo\\.eval_";
        ]
      }
    ]
  }
}
```

What this does:

1. Inspect every bash command for the string `aidi_catalog.dbo.eval_runs`.

2. If found, check whether the environment variable `AIDI_EVAL_SESSION` is set to `1`.
3. If it isn't, block the command with a clear error message.

Now an analyst exploring AKI cases can't accidentally `UPDATE` the eval table because they typoed a filter. An automation script that's *supposed* to write to `eval_runs` sets the environment variable up front and proceeds normally. The hook is invisible when behavior is correct and hard-stops when it isn't.

Pair this with the `aidi-eval-runner` subagent from Chapter 13: the subagent sets `AIDI_EVAL_SESSION=1` before its work and is the only path through which the eval table changes. Defense in depth.

 **Tip.** Pair hooks with explicit override mechanisms (an environment variable, a flag file) rather than making them unconditional. Unconditional hooks frustrate users into disabling them. A clear override path keeps the hook on permanently while still permitting deliberate work.

The `Stop` Hook (A Preview)

We'll cover the Ralph loop properly in Chapter 18, but the seed is here. A `Stop` hook fires when Claude finishes its turn:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "if [ -s TODO.md ]; then echo 'Tasks remain in TODO.md. Continue.'; fi"
          }
        ]
      }
    ]
  }
}
```

When the hook prints to stdout, that output is fed back into Claude's context as a new turn. So if `TODO.md` has any content, Claude doesn't stop; it sees a new instruction telling it to continue. That's a Ralph loop in seven lines of shell. The version that runs all night is more elaborate, but the mechanism is exactly this.

 **Remember.** Stop hooks turn Claude from a chat session into an autonomous worker. The agent only truly stops when the hook agrees there's nothing left to do.

When to Reach for a Hook vs. a Rule

- **Rule in `CLAUDE.md`** : anything Claude can be expected to follow voluntarily and where the consequence of a slip is "minor annoyance." Use conventions, naming, style, default model choice.
- **Hook**: anything where a slip is costly, irreversible, or invisible. Formatting (because catching unformatted code in review is friction). Destructive command blocking (because `rm -rf` doesn't have an undo). Audit logging (because you can't fake what wasn't recorded). Eval-table protection (because corrupted baselines hide future regressions).

 **Warning.** Don't put every rule in a hook. Hooks are powerful and noisy. The more hooks you have, the harder Claude is to work with. Reserve them for the non-negotiables. Everything else stays in `CLAUDE.md`.

Where Settings Live

- `.claude/settings.json` : project-level. Checked into git. Shared with the team.
- `.claude/settings.local.json` : project-level, gitignored. Your personal overrides for the project.
- `~/.claude/settings.json` : user-level. Applies everywhere.

Hooks compose across the three: a user-level hook fires on every project, a project-level hook fires only there, and a local file lets one developer add hooks without inflicting them on the team.

Try This

1. Open `.claude/settings.json` (create it if it doesn't exist). Add the Prettier or `ruff format` hook above, adapted to your language. Make Claude edit a file. Watch the formatting happen without you asking.
2. Add the destructive-command blocker. Then explicitly ask Claude to `rm -rf` something benign in a sandbox. Confirm the hook blocks it. Read the rejection message.
3. Add an audit log hook. Work for an hour. Then `cat .claude/edit-log.txt` and see exactly what got touched.

Watch Out

- **A bad hook can wedge your session.** A `PreToolUse` hook that always exits non-zero will block every action. If Claude seems frozen, check your hooks.
- **Hooks run with your shell's permissions.** Don't write a hook that does anything you wouldn't run yourself. There's no sandbox.
- **Regex blocklists leak.** A blocklist for `rm -rf /` is bypassed by `cd /; rm -rf .`. Treat the blocklist as a tripwire, not a fortress. Layer with permission settings and `/sandbox` when stakes are high.
- **Stop hooks can loop forever.** If your `Stop` hook always prints something, Claude never stops. Always have a termination condition: empty TODO list, max iterations, time of day.
- **Don't commit secrets via hook environment variables.** If your hook reads `$AZURE_OPENAI_KEY`, don't echo it. Don't log it. Don't pipe it through anything that might end up in `.claude/edit-log.txt`.

PART IV

Pushing the Limits

Chapter 15: MCP Servers

Your AIDI pipeline lives in three places (Azure SQL, a Databricks notebook, and a Slack channel where the on-call pharmacist asks why last night's AKI alert was empty) and Claude Code, by default, can only see the third of those if you paste it in by hand. MCP servers are how you fix that.

What MCP Actually Is

MCP stands for **Model Context Protocol**. It is an open standard, originally published by Anthropic, that lets an AI agent talk to an external tool or data source through a small, well-defined wire format. The agent says "list the tools you offer," the server replies with a menu, and from then on Claude can call those tools the same way it calls `Read` or `Bash`.

In practice, an MCP server is a tiny program (sometimes a single Node script, sometimes a Docker container) that sits between Claude Code and something Claude can't otherwise reach. Airtable, Notion, Slack, Postgres, Figma, Playwright, GitHub, Jira, your own internal API. Each one ships a server. You install it once. Claude treats it like a built-in.

 **Remember.** MCP doesn't make Claude smarter. It makes Claude's *reach* bigger. The same model that can edit a Python file can now also query your production database, read a Slack thread, or click a button in a real browser, because an MCP server exposed those actions as tools.

Installing One

The command is short:

```
claude mcp add <name> <command-or-url>
```

For a local server that runs as a subprocess:

```
claude mcp add playwright npx @playwright/mcp@latest
```

For a remote server reachable over HTTP:

```
claude mcp add aidi-sql https://mcp.khcc.jo/aidi-db
```

Once it's added, `claude mcp list` shows what's installed and `/mcp` inside a Claude session shows which servers are connected and which tools they expose. Removing one is `claude mcp remove <name>`.

 **Tip.** MCP servers can be installed at three scopes: local (this directory), user (every project on this machine), or project (shared via `.mcp.json` checked into git). Use project scope when your whole team needs the same connection (like Azure SQL) and user scope for personal stuff like your Notion workspace.

The Four to Install First

There are hundreds of MCP servers in the wild. Most of them you do not need. The community consensus (Suryansh's #24) is that four cover 80% of the real-world value:

1. **Playwright:** gives Claude a real browser. After Claude builds a UI, it can click through it, take a screenshot, and *verify* the button actually does what the user asked. This single capability moves Claude from "writes code that compiles" to "writes code that works."
2. **Postgres / MySQL / SQL Server:** Claude can describe your schema, run `SELECT` queries, and explain a slow plan, without you pasting `\d+ patients` into chat every time. For KHCC, this is the big one.
3. **Slack:** Claude reads the bug thread directly. The pharmacist's complaint, the timestamp, the screenshot — all of it becomes context without manual copy-paste.
4. **Figma:** for teams that ship UI. Claude reads the design file and writes the component to match. Design-to-code in one hop.

 **Remember.** Four MCP servers, used well, beats forty installed and forgotten. Every connected server adds tool definitions to Claude's context window. More servers = less room for your actual work.

When CLI Tools Beat MCP Servers

Here is the contrarian rule, and it is one of the most useful in this book: **a CLI tool is almost always more context-efficient than an MCP server that does the same job.**

Why? An MCP server registers its tools with Claude at session start. Every tool gets a name, a description, a schema. Even if you never call them, they sit in the context window costing tokens. Ten MCP servers with eight tools each is eighty tool definitions Claude must read on every turn.

A CLI tool, by contrast, costs zero tokens until Claude actually runs it. Claude already knows how to call `gh`, `git`, `kubectl`, `aws`, `az`, `sentry-cli`, `psql`, and `curl` from training, and if it doesn't, it can run `--help` once and learn.

So:

- **Need to open a GitHub PR?** Don't install a GitHub MCP server. Teach Claude to run `gh pr create` and give it the auth token.
- **Debugging a production crash?** `sentry-cli events list --org khcc` is one command. The Sentry MCP server is forty.
- **Querying a Postgres dev database?** `psql -c "SELECT ..."` works fine. The Postgres MCP shines when you need *stateful* exploration of schema you don't know.

🔧 **Technical Stuff.** The context cost difference is real and measurable. Run `claude mcp list` after installing five servers and watch `/cost` on your next session. You will see the per-turn input tokens climb by 2,000–6,000 just from registered tool definitions.

When MCP Actually Wins

MCP is the right answer when:

- **The tool has no good CLI.** Figma is a design app, not a command. Playwright has a CLI but exposing browser state to an LLM through it is awkward; the MCP server is the right interface.
- **The integration is stateful.** Notion pages, Slack threads, Airtable bases: these have IDs, relations, and pagination that benefit from a typed tool surface.
- **You want type-safe arguments.** An MCP tool with a JSON schema is harder for Claude to call wrong than a bash command with positional args.
- **The tool is shared across your team.** A `.mcp.json` checked into your repo gives every teammate the same `aidi-sql` connection on `claude mcp add`. There is no equivalent for "we all use the same shell aliases."

⚠️ **Warning.** Every MCP server you install can read whatever it's connected to. A Postgres MCP pointed at production can `DROP TABLE`. A Slack MCP can post to any channel your token has access to. Treat MCP credentials like database credentials, because that's what they are. Never install a third-party MCP server from a random GitHub repo without reading the code.

The KHCC Example

Here is the AIDI setup that earns its keep.

An MCP server for Azure SQL `AIDI-DB`. A small Python server, running on the AIDI bastion, exposes three tools to Claude: `list_tables`, `describe_table`, and `run_select` (read-only, hard-coded to reject anything that isn't a `SELECT`). Now when an analyst asks Claude "build me a cohort of bone-only metastatic breast cancer patients from the last three years," Claude doesn't have to guess

that the table is `GOLD_DIAGNOSIS` and the column is `met_site_bone`. It calls `describe_table GOLD_DIAGNOSIS`, sees the schema, writes the right query, runs it through `run_select`, sanity-checks the row count, and only then asks you to approve the final SQL. No DBA paged. No schema docs out of date.

An MCP server for the KHCC Optimus encoding service. The Optimus encoder turns a real MRN into the obfuscated ID that gets logged, emailed, and dashboarded. Without an MCP, Claude either hard-codes an MRN (a privacy violation waiting to happen) or asks you for the encoded form (a workflow break). With the MCP, Claude calls `optimus_encode "<mrn>"`, gets the safe ID back, and uses it in the script. The real MRN never leaves the encoder process.

Both of these are about 200 lines of Python each. They sit in `aidi-mcp-servers/` in the AIDI repo. Every analyst who pulls the repo and runs `claude mcp add aidi-sql ...` gets the same connection.

 **Tip.** When you write your own MCP server, give every tool a description that tells Claude *when not to use it*. "Runs a read-only SELECT against AIDI-DB. Use only for cohort queries. Do not use to count rows; use `describe_table` for size estimates." Claude follows tool descriptions far more reliably than it follows your prompts.

Try This

1. Run `claude mcp list` to see what (if anything) is already installed.
2. Install one. Playwright is the most fun. `claude mcp add playwright npx @playwright/mcp@latest`.
3. In a Claude session, ask: "Open google.com, search for 'KHCC', and tell me the title of the first result." Watch Claude drive a real browser.
4. Now uninstall it: `claude mcp remove playwright`. Compare the `/cost` of a one-turn session before and after. That delta is what every always-installed MCP costs you.

Watch Out

- **Don't install five servers before you've used one.** Tool definitions are context tax. Pay only for the ones you use this week.
- **Read the source of any third-party MCP server before connecting it to a database.** "Open source" is not "safe."
- **Read-only by default.** Your AIDI-DB MCP should refuse anything but `SELECT`. The day someone writes `DELETE FROM patients WHERE ...` in a prompt is the day you learn this the hard way.
- **CLI first, MCP second.** If `gh`, `psql`, or `az` will do the job, use them. Save MCP for tools that genuinely need a typed interface.

Chapter 16: Worktrees and Parallel Clauses

You're halfway through a prompt-tuning session on the pathology extractor when the chemotherapy dashboard catches fire and an analyst asks you to look at the AKI cohort SQL "real quick." Three tasks, one repo, one Claude. Somewhere in the next ten minutes you will paste the wrong file into the wrong context and waste an hour. Git worktrees are how you stop doing that.

What a Worktree Actually Is

A **git worktree** is a feature git has had since version 2.5, and almost nobody uses. It lets one git repository have *multiple working directories* at once, each checked out to a different branch, sharing the same `.git/` object store underneath.

In plain terms: instead of `git checkout feature-auth` (which destroys your current uncommitted state on `main`), you run `git worktree add ../aidi-feature-auth feature-auth` and now you have a second folder on disk, with the same repo, on a different branch, that you can open in a second terminal. Two folders. Two branches. One repo. Zero conflicts.

 **Remember.** A worktree is not a clone. Cloning duplicates history, takes minutes, and loses your local config. A worktree is instant, shares all history, and is meant to be thrown away.

Why This Matters for Claude Code

Claude Code, by default, operates in whatever directory you launched it from. One terminal, one Claude, one branch. If you open a second terminal and run `claude` in the same folder, you now have two Claudes editing the same files: a race condition with opinions.

Worktrees fix it cleanly. One worktree per task = one Claude per task = no interference.

The `--worktree` Flag

Claude Code has a built-in shortcut for the whole flow:

```
claude --worktree feature-auth
```

or the short form:

```
claude -w feature-auth
```

This does three things in one shot:

1. Creates a new git worktree in a sibling directory.
2. Checks out (or creates) the branch `feature-auth`.
3. Launches Claude with that worktree as its working directory.

When you exit that Claude session, if the worktree has **no uncommitted changes and no new commits**, it gets cleaned up automatically: the directory removed, the branch left intact if it had commits, deleted if it didn't. If you made changes, the worktree stays around until you deal with it manually.

 **Tip.** `git worktree list` shows every worktree currently attached to the repo. `git worktree remove <path>` cleans one up. `git worktree prune` cleans up references to worktrees whose directories you deleted manually.

Three Terminals, Three Clauses

The real power move: open three terminals. In each one, launch Claude with a different worktree.

```
# Terminal 1
claude -w aki-cohort-sql

# Terminal 2
claude -w pathology-prompt-tweak

# Terminal 3
claude -w dashboard-refactor
```

Three Clauses, three branches, three working directories. They cannot step on each other because they are literally in different folders. You can task each one and rotate attention between them as they think.

 **Technical Stuff.** Each worktree gets its own untracked-file state and its own `.claude/` configuration if you set one per-worktree. Hooks, slash commands, and skills inherit from the user-scope configuration; project-scope `CLAUDE.md` is shared because it lives in the tracked tree. This is usually what you want.

Subagents Get Their Own Worktrees Automatically

This is the under-advertised feature. When Claude spawns a subagent (Chapter 13), the subagent runs in *its own* worktree by default. So if your main Claude delegates "refactor the lab-reconciliation pipeline" to a subagent, that subagent gets a clean worktree, does its work, and reports back the diff, without ever touching the file you're editing in the main session.

This is why agent teams (Chapter 13) actually work in practice. Five subagents would be chaos if they all wrote to the same directory. Five worktrees, one per subagent, is just five normal Claude sessions running in parallel.

 **Remember.** Worktrees are what make parallel Claude work safe. Without them, "two agents on one repo" is a foot-gun. With them, it's a force multiplier.

When to Use a Worktree (and When Not To)

Use a worktree when:

- You have two or more genuinely independent tasks and want them in parallel.
- You're starting an experiment you might throw away and don't want to pollute your main branch.
- You're doing a long-running refactor on one branch and need to ship a hotfix on another.
- You're running a Ralph loop (Chapter 18) overnight on one task and want to keep working on another in the morning.

Don't bother when:

- The task is a one-line fix. Worktree setup is faster than your terminal startup, but `git stash` is still faster.
- You're exploring a single repo with no intent to branch. Just use `main`.
- You're new to git and worktrees feel confusing. Get fluent with branches first; worktrees will make sense the moment branching does.

 **Warning.** A worktree is a real directory on disk with real uncommitted changes. If you delete a worktree directory manually (`rm -rf`) while it has uncommitted work, that work is gone. Use `git worktree remove` instead; it refuses to delete dirty worktrees.

Merging Back

When a worktree's work is ready to land, the flow is unremarkable:

```
cd ../aidi-feature-auth      # the worktree
git push -u origin feature-auth
gh pr create                 # if you use GitHub
# or merge locally:
cd ../aidi                   # back to main worktree
git merge feature-auth
git worktree remove ../aidi-feature-auth
git branch -d feature-auth   # if you're done with the branch
```

Same as branches you'd merge any other way. The only worktree-specific step is the cleanup at the end.

The KHCC Example

It's Tuesday afternoon. You have three things on your plate:

1. **New AKI cohort SQL.** A surgeon asked for a cohort of post-op AKI cases stratified by procedure type. You haven't written the cohort query yet.
2. **Pathology extraction prompt tweak.** The eval run last night flagged a regression on T-stage extraction for breast cases. Needs a prompt change and a re-run.
3. **Dashboard refactor.** The chemo-prep dashboard's Streamlit code has been on your conscience for two weeks. You want to extract the patient-card component into a reusable widget.

You could try to do all three sequentially. You could try to do them in one Claude session and watch the context window meltdown by task #2. Or:

```
claude -w aki-cohort-sql
# Terminal 2:
claude -w path-prompt-fix
# Terminal 3:
claude -w dashboard-refactor
```

Now you have three Claudes thinking in parallel. Claude #1 is reading `aidi-extractions/` to learn the AKI cohort conventions. Claude #2 is loading the frozen deceased-patient eval cohort and looking at the T-stage mismatches. Claude #3 is mapping out the Streamlit component split.

You start them in that order, then circle back to Claude #1 first, review the cohort SQL it drafted, kick off a sample query, then move to Claude #2 while #1 runs. By the end of the afternoon all three branches have draft PRs and you've context-switched between them about six times. None of them touched the other's files. None of them got confused about which branch they were on. None of them had a conflict.

This is what people mean when they say Claude Code multiplies throughput. Not "Claude writes faster than you." Claude doesn't. What multiplies throughput is *three Claudes running while you supervise one at a time*.

 **Tip.** Name your worktrees the same as the branch they hold. `aki-cohort-sql` (worktree dir) holds branch `aki-cohort-sql`. When you have eight worktrees open and need to remember what each one does, `git worktree list` will read like a to-do list.

Try This

1. In an existing git repo, run `git worktree add ../experiment experiment-branch`. Open the new directory in your file explorer. Confirm it looks like a normal checkout.
2. In your original repo, edit a file but don't commit. Switch to the worktree. Notice your uncommitted change is **not** there: each worktree has its own working tree state.
3. Now exit and try `claude -w try-worktree`. Make no changes, exit. Confirm the worktree was auto-cleaned.
4. Open three terminals, three worktrees, three tasks. Notice how the cognitive load of "which branch am I on" disappears.

Watch Out

- **Don't share a worktree across two Claude sessions.** That defeats the whole point. One worktree, one Claude.
- **Don't worktree everything.** Three is a sweet spot. Eight worktrees is eight Claudes you can't supervise.
- **Clean up.** `git worktree list` should not have entries from three months ago. Stale worktrees rot.
- **Auto-cleanup only happens on a clean exit.** If you `kill -9` Claude, the worktree stays. Run `git worktree prune` periodically.
- **CLAUDE.md is shared across worktrees.** A change you make in one worktree's `CLAUDE.md` will appear in the others as soon as you commit and merge. Don't put per-task instructions there.

Chapter 17: Headless Mode and Automation

Claude Code is, on the surface, a chat program. You type, it thinks, it asks permission, you approve, it edits. That is one mode. There is another mode where Claude is a Unix tool, no chat, no prompts, no terminal at all, just a process that reads input, does the work, and writes output to stdout. That mode is how you put Claude inside a cron job, a GitHub Action, or a nightly pipeline.

The `-p` Flag

The whole feature is one flag: `-p` (short for "print"). It used to be called *headless mode* in the early docs. The newer documentation calls it *CLI mode* or *non-interactive mode*. Same feature.

```
claude -p "fix the failing tests in this repo"
```

When you run that, Claude does **not** open the interactive UI. It reads the prompt, runs the agentic loop until it thinks it's done, prints the final answer to stdout, and exits with a status code. No prompts. No approval dialogs. No "are you sure?" because there is no human to ask.

That last part is the whole point and also the whole danger.

 **Remember.** `-p` turns Claude from a *conversational tool* into a *CLI tool*. It's what you reach for when there is no human in the loop. Anything you wouldn't trust to run without supervision should not go through `-p`.

Allowlists Are Mandatory

Because there is no human to approve actions, you must pre-approve them. The flag is `--allowedTools`:

```
claude -p "summarize today's pipeline errors" \
  --allowedTools "Read,Grep,Bash(grep:*)"
```

This says: Claude may read files, grep them, and run `grep` from bash, but nothing else. Try to write a file? Denied. Try to `rm -rf`? Denied. Try to `curl evil.com`? Denied.

The opposite flag is `--disallowedTools`, which inverts the logic (allow everything except these). I do not recommend it. **Allowlist surgically; never blanket-allow.**

⚠ Warning. Combining `-p` with `--dangerously-skip-permissions` (the nuclear flag from Chapter 4) is how people lose data overnight. Even Boris Cherny, who built Claude Code, says he never uses skip-permissions. In headless mode it is doubly true. If you find yourself reaching for it, you have under-specified your allowlist. Fix the allowlist instead.

Piping In, Piping Out

Because `-p` reads stdin and writes stdout, it composes with everything Unix already gives you:

```
# Pipe a log file in, get an explanation out
cat error.log | claude -p "explain this stack trace and suggest a fix"

# Pipe a diff in, get a review out
git diff main | claude -p "review this change for SQL injection risks"

# Pipe Claude's output into another tool
claude -p "list every TODO in this repo" --allowedTools "Read,Grep" | tee todos.txt
```

This is the Unix philosophy applied to an AI agent: small programs that read text and write text, glued together with pipes.

💡 Tip. Combine `-p` with `--output-format json` to get structured output you can parse in a script. The shape includes the assistant's final message, token counts, and any tool calls made. Perfect for logging or for feeding into a second pipeline stage.

CI/CD Integration

The poster-child use case is a GitHub Action that reviews every PR. The Action checks out the PR, runs:

```
claude -p "Review the diff in this PR for bugs, security issues, and KHCC clinical-pipeline c
--allowedTools "Read,Grep,Bash(gh:*)"
```

Claude reads the diff, runs `gh pr view` and `gh pr diff` to get context, evaluates the change, and uses `gh pr review --comment` to leave findings. No human sits at a console. The Action runs in seconds-to-minutes per PR.

The same shape works for:

- **Nightly migration sanity checks.** A cron job runs Claude against last night's ETL output and writes a report.

- **Dependabot triage.** When a bot opens a PR bumping a dependency, Claude reads the changelog and flags whether the bump is safe to auto-merge.
- **Stale-issue triage.** A weekly job reads every issue older than 60 days and labels them.

 **Technical Stuff.** Inside a GitHub Action, the `ANTHROPIC_API_KEY` environment variable is what Claude Code looks for. Store it as a repo secret. Never echo it into logs. The Action `actions/setup-node` plus `npm install -g @anthropic-ai/claude-code` is the whole install dance.

Cron Jobs and Scheduled Tasks

On Linux, `cron`. On macOS, `launchd`. On Windows, Task Scheduler. The shape is identical: at a fixed time, run a script. That script invokes `claude -p`.

```
# /etc/cron.d/aidi-nightly
0 6 * * * aidi-bot /opt/aidi/scripts/nightly-summary.sh >> /var/log/aidi-claude.log 2>&1
```

The `nightly-summary.sh` script is just bash glue around a `claude -p` call. The log captures stdout. The exit code captures pass/fail.

 **Remember.** Cron + `-p` is how Claude Code goes from "an interactive tool I open" to "a service that runs whether I'm at my desk or not." This is the leap from coding agent to teammate.

The Safety Math

Here is the rule I want you to commit to memory:

Headless × surgical allowlist = safe automation. Headless × skip-permissions = uncontrolled execution.

Every `-p` invocation should look like a recipe: a tight prompt, a tight allowlist, a tight output destination. The day you find yourself writing `claude -p "..." --dangerously-skip-permissions` in a cron job is the day to step back and ask which specific tools the job actually needs.

 **Warning.** A headless Claude that can write files and run bash, scheduled to run at 3 AM, is functionally indistinguishable from a malicious script you wrote on purpose. Treat the allowlist like a firewall rule. Default deny.

The KHCC Example

Here is the nightly job that runs on the AIDI bastion at 6:00 AM every weekday.

A new VistA extract lands in `/data/vista/incoming/` around 5:30 AM. The AIDI team wants a one-page summary every morning before standup: row counts, new MRN counts, any unusual nulls or schema drifts compared to the previous day, and a flag if any column unexpectedly went 100% null (which usually means an upstream rename).

The script:

```
#!/bin/bash
# /opt/aidi/scripts/vista-morning-summary.sh

set -euo pipefail
cd /data/vista/incoming

claude -p "Compare today's VistA extract files to yesterday's. \
  For each file: report row count delta, new column count, columns whose null rate \
  changed by more than 5 percentage points. Flag anything that looks like a schema rename. \
  Output as markdown. Then send the markdown body to the AIDI team list via the \
  'aidi-email' CLI tool." \
  --allowedTools "Read,Grep,Bash(ls:*),Bash(wc:*),Bash(head:*),Bash(aidi-email:*)" \
  --output-format text \
  > /var/log/aidi/vista-summary-$(date +%F).md
```

Tight allowlist: read files, grep them, run a few read-only bash commands, send one email through the AIDI internal mailer. Cannot write to the data directory. Cannot delete. Cannot connect to anything but the mailer. Cannot run arbitrary bash.

By 6:05 AM the AIDI team has a markdown summary in their inbox. The on-call data engineer sees it on her phone before she's at her desk. If the summary is empty or the script failed, the cron log shows it. If there's a real schema drift, the summary surfaces it before any downstream pipeline runs.

 **Tip.** For any new headless job, run it interactively (without `-p`) first, watching what tools Claude actually reaches for. The set of tools it used is your allowlist. Don't guess.

Try This

1. In a sandbox folder with a couple of files, run: `bash echo "list every TODO comment" | claude -p --allowedTools "Read,Grep"` Confirm you get a clean text answer with no UI prompts.
2. Add `--output-format json` and pipe through `jq .` to see the structured response. Note the `total_cost_usd` field: this is how you budget cron jobs.

3. Write a one-line cron job that runs `claude -p "summarize what changed in this repo in the last 24 hours"` and writes the output to a file. Read it the next morning.
4. Now imagine that job ran with `--dangerously-skip-permissions` and a buggy prompt. That thought should make you uncomfortable. Good.

Watch Out

- **No human = no recovery.** A headless run that goes off the rails will keep going off the rails until it exits, hits an allowlist wall, or burns its turn limit. Set `--max-turns` (e.g. `--max-turns 20`) for jobs that should be short.
- **Costs add up.** A nightly job that runs Claude with Opus on a 200-file repo can be \$5–\$20 per run. That's \$150–\$600/month for one nightly job. Use Sonnet for routine jobs; reserve Opus for ones where quality genuinely matters. Check `/cost` or the JSON output.
- **Secrets in prompts.** Anything you put in the `-p` prompt or in files Claude reads is sent to Anthropic. Don't embed PHI, MRNs, or API keys. For KHCC pipelines, encode MRNs with Optimus *before* anything reaches the prompt.
- **Surgical allowlists, every time.** If you find yourself blanket-allowing `Bash`, you're one prompt injection away from a bad day.

Chapter 18: Running for Hours (The Ralph Loop)

Meter, the AI evaluation lab, recently published a benchmark showing that Claude Opus 4.5 can perform autonomously on real software-engineering tasks for **4 hours and 49 minutes** at a 50% completion rate. For context: when GPT-4 launched, the equivalent number was about five minutes. We have entered an era where coding agents can run for the length of a workday, and this chapter is about how to actually use that.

The Boris Cherny Tweet

Boris Cherny is the creator of Claude Code. In late 2025 he posted a tweet that made the rounds:

"When I created Claude Code as a side project back in September 2024, I had no idea it would grow to what it is today... Fast forward to today, the last 30 days, I landed **259 PRs, 457 commits, and 40,000 lines added, and 38,000 lines removed**. Every single line was written by Claude Code and Opus 4.5. Claude consistently runs for minutes, hours, and days at a time using **stop hooks**. Software engineering is changing."

Read that paragraph twice. The author of Claude Code lands roughly nine PRs per day, every day, with **zero hand-written lines**. The mechanism that makes it possible is one feature: the **stop hook**.

 **Remember.** Boris's productivity is not about a magic prompt. It is about a deterministic harness around the non-deterministic model. The model writes the code. The harness keeps the model on task.

Why Prompting Alone Goes Lazy

If you try to extract long-running behavior with prompts alone ("do all twenty of these tasks, don't stop, run tests after each") Claude will get partway through, decide it's done, and exit. Not because it's lazy in the human sense, but because the model is trained to terminate when it thinks the user's goal is met. After task seven of twenty, the conversation looks "complete enough" and out it goes.

You cannot prompt your way out of this. You need a mechanism *outside* the LLM that fires when Claude tries to stop, checks whether the work is actually done, and if not, feeds the next instruction back in.

That mechanism is the **stop hook**.

Stop Hooks, Briefly

Chapter 14 introduced hooks: shell commands that fire at specific points in Claude's workflow. The **Stop** event fires when Claude is about to return control to you and exit the agentic turn. A stop hook is a shell command attached to that event.

Two things a stop hook can do that no prompt can:

1. **Block the stop.** If the hook exits non-zero, Claude does not return to you. The turn continues.
2. **Feed in new context.** Whatever the hook writes to stdout becomes part of the next prompt cycle. So the hook can say "check the to-do list: items 4, 5, and 7 are still unchecked. Keep going."

That is the whole trick. Block the exit. Inject the next instruction. Repeat until done.

 **Technical Stuff.** The hook gets the session's recent state on stdin (as JSON) and can return JSON on stdout instructing Claude what to do next. The full schema is in the Claude Code docs under `Hooks` → `Stop`. For most uses, a 20-line shell script is enough.

The Ralph Loop

The pattern has a name: the **Ralph loop**, after Ralph Wiggum from *The Simpsons*, the kid whose entire personality is "I'll keep trying." Boris named it. Anthropic ships an official plugin called `ralph-wiggum` that bundles the stop hook, a state file, and a `/ralph-loop` slash command.

The shape:

1. You give Claude a prompt and a **completion promise**: a sentence describing what "done" means. ("All items in `todo.md` are checked off and `pytest` exits zero.")
2. You set a **max iterations** cap. (Common: 20–50.)
3. You kick it off.
4. Claude works on the task. When it tries to exit, the stop hook fires.
5. The hook checks the completion promise. Not met? Re-injects the prompt with a "keep going, here's what's left" suffix.
6. Loop until either the promise is met or max iterations is hit.

Inside that loop, Claude is doing what Claude always does: reading files, editing code, running tests. The only difference is it cannot quit until the harness lets it.

 **Remember.** Two guardrails are non-negotiable: **max iterations** and a **completion promise**. Without max iterations, a bug becomes an infinite loop that bills your card. Without a completion promise, the hook has no way to know when to stop, and it won't.

The To-Do File Pattern

The most reliable Ralph setup uses a markdown to-do file as the source of truth:

```
# refactor_todo.md

- [ ] Move AKI staging logic from notebook A to shared module
- [ ] Update pipeline B to import from shared module
- [ ] Add unit test for AKI stage 2 boundary case
- [ ] Run eval cohort and confirm alert count matches baseline
- [ ] Update CLAUDE.md to point to the new module
```

You prompt Claude something like:

"Work through `refactor_todo.md` step by step. For each unchecked item: do the work, run the relevant tests, and only mark `[x]` when tests pass. Then move to the next. Do not stop until every item is checked."

The stop hook re-reads `refactor_todo.md` after every turn. If it finds an unchecked box, it blocks the stop and re-injects the prompt. If everything is checked, it lets Claude exit.

This is brilliant because the state lives in a *file*, not in Claude's head. Claude can forget what it was doing, lose context, get compacted, and the next iteration just re-reads the to-do file and continues from wherever it left off.

 **Tip.** Always include a validation step in each to-do item: a test, an eval run, a row-count check. Without it, Claude can mark items complete that aren't, and you'll come back in the morning to a fully-checked list and a broken pipeline.

Overnight Refactors and Migrations

The Ralph loop's sweet spot is work that is:

- **Long.** Hours, not minutes.
- **Listy.** Decomposable into a flat list of similar tasks.
- **Verifiable.** Each task has a deterministic pass/fail check (a test, a query, a file existence check).

Examples that work beautifully:

- Refactoring fifty notebooks to use a shared helper.
- Migrating a codebase from one library version to another.
- Updating every Markdown file in a docs site to a new style guide.
- Re-prompting twenty AIDI extraction pipelines and re-running their eval cohorts.

Examples that do **not** work well:

- Open-ended research ("figure out the best architecture for X"). No completion check exists.
- Single-file feature work. Too small; just supervise it.
- Anything where each task depends on a human decision midway.

The Three Dangers

There are exactly three ways a Ralph loop ruins your day. Memorize them.

1. Infinite loop. No completion promise, or a promise the hook can't actually check. The loop runs until you notice. By then it has burned an Opus subscription's worth of tokens.

 **Warning.** Always set `--max-iterations 30` (or whatever fits your task). Even with a completion promise, this is your circuit breaker.

2. Token burn. Even a well-shaped loop running on Opus across a large codebase can spend \$50–\$200 in a night. That can be money well spent. It can also be money wasted because you misjudged the prompt. Sanity-check the per-iteration token usage on a short run before kicking off the long one.

3. Cascading silent failures. Without per-iteration validation, Claude can build on top of subtly wrong work. By iteration fifteen, the codebase looks "done" but is wrong in ways the to-do checks miss. This is why every to-do item needs a real test, not just a "Claude believes it works" check.

Combining With Worktrees and Subagents

The full-power configuration:

- 1. One worktree per Ralph loop.** Run the loop in `~/aidi-prompt-refresh/` while you keep working on main features in the primary repo. (Chapter 16.)

2. **Subagents for parallel sub-tasks.** Within a Ralph iteration, Claude can spawn subagents that each refactor one notebook, run in parallel, and report back. (Chapter 13.)
3. **Headless `-p` mode for full unsupervised runs.** Combine `-p`, a tight allowlist, a Ralph loop plugin, and a max-iterations cap. (Chapter 17.) You can launch this from a cron job and walk away.

 **Remember.** Worktrees + subagents + Ralph + headless is the configuration Boris Cherny uses. It is not magic. It is four features stacked, each one earning its keep, each one with a guardrail.

The KHCC Example

It is Friday at 5:30 PM. You have a `to-do` list of twelve AIDI extraction pipelines that need their prompts updated for the new GPT-4.1-mini deployment and re-evaluated against the frozen deceased-patient cohort. Each one takes maybe 20 minutes of supervised work. Twelve $\times$ 20 = four hours. You don't want to stay until 9:30 PM.

You write `friday_refresh_todo.md`:

```
# Friday Prompt Refresh

For each pipeline:
- Update the prompt to use the new gpt-4.1-mini system message template.
- Re-run the eval against the frozen deceased-patient cohort.
- Confirm eval pass rate is >= prior baseline (stored in eval_runs).
- Mark the item complete only if eval passes.
- If eval regresses, leave the item unchecked and add a note explaining why.

- [ ] pathology_v6_2
- [ ] er_extractor
- [ ] aki_notification
- [ ] chemo_prep
- [ ] vanco_tdm
- [ ] amr_audit_breast
- [ ] amr_audit_urology
- [ ] amr_audit_ent
- [ ] discharge_summary
- [ ] bmt_gvhd_extraction
- [ ] readmission_risk
- [ ] icu_dx_extract
```

You make a fresh worktree, launch Claude with the Ralph plugin, set `--max-iterations 40` and a completion promise of "every item in `friday_refresh_todo.md` is checked or annotated, and `aidi_catalog.dbo.eval_runs` has a new row for each pipeline."

You go home.

Saturday morning you check the worktree. Eight items checked. Three items unchecked with notes explaining mild regressions. One item flagged "needs human review: eval pass rate dropped from 0.91 to 0.74, prompt may need restructuring." Total runtime: 5 hours 12 minutes. Total cost: \$34. Total of your evenings burned: zero.

Monday you spend an hour on the four flagged items. The refresh that would have taken you a week is done by Tuesday lunch.

This is what the new era of coding agents actually looks like. Not Claude replacing you. Not Claude doing the easy 80% poorly. Claude doing the listy, repetitive, verifiable 80% correctly, overnight, while you sleep, while you spend Monday on the four interesting cases where judgment is genuinely needed.

 **Tip.** Always have the Ralph loop write a per-iteration log file. When you review on Monday, you want to see *why* each item was marked or unmarked, not just the final state. A loop without an audit trail is hard to trust.

Try This

1. Install the `ralph-wiggum` plugin (`claude plugins add ralph-wiggum` ; exact command may vary by version, check the docs).
2. Make a `practice_todo.md` with five tiny tasks (create file, rename function, add a docstring, etc.) and a check for each.
3. Run `/ralph-loop` with a completion promise of "every item is checked off and tests pass" and `--max-iterations 15` .
4. Watch the stop-hook fire between iterations. Notice that Claude doesn't get to leave until the list is done.
5. Then deliberately add an impossible item ("connect to a server that doesn't exist") and watch the loop spend its iteration budget trying. This is why max iterations exists.

Watch Out

- **Never run a Ralph loop without max iterations.** This is the single most expensive mistake in this book.
- **Never run one without a completion promise the hook can actually verify.** "Until it's good" is not a promise. "Until pytest exits zero" is.
- **Validate per-item, not just at the end.** A loop that checks twenty items and only tests at the end will hide nineteen cascading failures.

- **Watch the cost.** Set a token budget and monitor `/cost` during the first short run. Multiply by your iteration count. If the math is alarming, change the model to Sonnet or shrink the prompt.
- **Don't Ralph-loop a clinical pipeline change without a real eval gate.** The eval suite against the frozen deceased-patient cohort is the only acceptable completion criterion for AIDI prompt changes. "Tests pass" is necessary but not sufficient. "Eval pass rate $\geq$ baseline" is the bar.
- **Ralph is not a substitute for thought.** It is a tool for executing a plan you already have. If you wouldn't trust a junior engineer to grind through your to-do list overnight, don't trust Claude to either; fix the to-do list first.

PART V

The Part of Tens

Chapter 19: Ten Killer Workflows for Clinical AI Teams

The rest of this book taught you the moving parts. This chapter shows you what to actually build with them on a Tuesday morning at a cancer center.

Every workflow below maps directly to something AIDI, pharmacy informatics, or the data science team at KHCC already does, or should be doing. Pick two. Try them this week.

1. The Nightly Extraction Eval Run

You change a prompt. You wonder if it broke anything. The deceased-patient eval cohort sits there waiting.

Wire up a Stop hook to a `/eval` slash command that runs your frozen eval suite against `aidi_catalog.dbo.eval_runs` and diffs against last night's baseline. Claude commits the prompt change, runs the eval, posts a markdown summary to the team Slack with delta counts ("AKI worsening: 142 → 141, one patient flipped: review case MRN A8X4-92"). Pair it with a Ralph loop and you can iterate on a prompt for an hour while you eat dinner.

 **Remember.** No prompt change ships without an eval run. That's Rule 4 from the global `CLAUDE.md`. Wire it into a hook so you can't forget.

2. The Pathology Triage Inbox

Wilms tumor cases land as PDFs. The clinician wants T-stage, histology, and margin status pulled out before clinic.

Build a small project: a `pathology-inbox/` folder Claude watches via a Stop hook, a Pydantic schema for the structured output, and a slash command `/process-inbox` that calls `client.beta.chat.completions.parse(...)` for each new file and writes a `.json` next to it. Add a Hook that runs `tools/check_pii.py` before any file moves out of the inbox folder.

3. The "Why Did This Patient Get Missed?" Auditor

A patient who should have triggered an AKI alert didn't. Three pipelines touch their record, two teams own them, and nobody wants to start the spelunking.

Spin up a subagent with read-only tools (`Read` , `Grep` , `Glob`) pointed at the AIDI extraction repo, the cohort SQL, and the alert dispatcher. Prompt: "MRN A8X4-92 should have triggered the AKI worsening alert on 2026-04-12. Trace every step. Report where the patient was filtered out and which table is responsible." Subagent context stays clean, main session stays clean, and you get a 200-word root cause in two minutes instead of two hours.

 **Tip.** Security at Anthropic shaved 10–15 minutes of stack-trace tracing down to ~5 minutes with this exact pattern. The pattern is real. The numbers are real.

4. The Two-Tool Plan-Then-Build Workflow

For anything bigger than one file, don't start in Claude Code. Start in Claude.ai.

Brainstorm the design in the chat app: trade-offs, schema, edge cases, what to *not* build. When you have a plan, ask Claude.ai to write you the implementation prompt for Claude Code. Paste that into Claude Code as your opening message. The Anthropic Growth Marketing team and the Legal team both discovered this independently. It works.

5. The Dashboard Screenshot Debug

A clinician slacks you a screenshot of the chemotherapy dashboard with three red boxes and the message "this is wrong." You don't know which "this."

Paste the screenshot into Claude Code. Say "the user reports the three highlighted rows are wrong; find the bug." Claude reads the image, identifies the columns, traces the query, finds the join error, fixes it. Data Infra at Anthropic uses this pattern to diagnose Kubernetes outages from GCP screenshots. It works on Power BI screenshots too.

 **Remember.** Images use context, but they're often cheaper than the 600 words you'd need to describe the bug in prose.

6. The Friday Ralph Loop

You have a `todo.md` with 17 unchecked items: refactor four prompts, regenerate three figures, update a Cox model, write the methods section, run three more evals. You also have a flight to catch.

Set up the Ralph loop with a `completion_promise` that requires every checkbox ticked and the eval suite green. Set `max_iterations` to 30. Launch. Get on the plane. By the time you land, most of it's done, the failed items are flagged with explanations, and you owe somebody a coffee but not a weekend.

7. The R Survival Analysis Buddy

You have a `survminer` chart that needs Lancet styling: stratified by treatment arm, p-value box, risk table, color palette from the published figure.

Open Claude Code in the project folder. Paste the target figure as an image. Say "match this with our `tidymodels` cohort, theme it for Lancet, save as PNG and PDF." Claude reads `theme_lancet.R` from your existing code, writes the new script, runs it, shows you the result. If wrong, paste the rendered PNG back in. Iterate until it matches. The whole loop is twenty minutes.

8. The MCP-Powered AIDI-DB Schema Inspector

Half your day is "wait, what's that column called in `SILVER_aki_episodes`?"

Install a Postgres/Azure SQL MCP server pointed at `AIDI-DB`. Once it's connected, you ask in plain English: "Which silver tables have a `creatinine_baseline` column, and which gold table aggregates them?" Claude queries `information_schema`, returns the answer, and remembers it for the rest of the session. The Anthropic Data Infra team replaced their data catalog with this exact pattern.

9. The Custom Slash Command Library

Every team has rituals: "make a commit message in the AIDI style," "open a PR against the dev branch with the standard template," "regenerate the patient cohort SQL with today's date." Each of these is one markdown file in `.claude/commands/`.

Spend a Friday afternoon writing eight of them. Check them into the repo. Now everyone on the team gets the same `/commit-aidi`, `/pr-dev`, `/refresh-cohort`. Security at Anthropic owns *half* of all custom slash commands in the entire company monorepo. They are the visible signature of a team that takes the tool seriously.

10. The Non-Coder Workflow Translator

A clinician comes to you with: "I want to pull yesterday's referral count, filter to oncology, group by service, and email the top five doctors."

You don't write the script. You teach them to write the *prompt*. Open Claude Code in a sandboxed folder, hand them the keyboard, watch them describe what they want in plain English. Claude writes the code, runs it, shows the result. You add a Hook that prevents anything destructive. The Anthropic Finance team trained their non-coders to do exactly this. The long-term value is not the script they got today; it's the dozen scripts they'll write next month without you.

 **Warning.** Sandbox aggressively for non-coders. `.claude/settings.json` deny-list everything in `/data/patients/` and any `*.key` / `*.env` file. The Hook chapter (14) covers the `PreToolUse` blockers you'll want.

Try This

Pick workflow #1 (the eval run) and stand it up before you read Chapter 20. It's the single highest-payoff automation in this list, and it touches every concept in the book: hooks, slash commands, subagents, CLAUDE.md. If you can ship #1, you can ship the other nine.

Watch Out

Don't try to build all ten in a week. Pick two. The trap with Claude Code is that everything is just-possible-enough that you'll over-commit. Two workflows shipped beats ten started.

Chapter 20: Ten Rookie Mistakes to Avoid

Most of these were learned by people, including the author, doing the dumb thing first.

1. Running `--dangerously-skip-permissions` on Day One

You'll see the `cc` alias trick on Twitter and feel left out. Don't. Boris Cherny, the creator of Claude Code, says he doesn't use it. You don't need to either, at least not until you've spent a week watching what Claude actually wants to do. Pre-approve specific safe commands with `/permissions` instead. The speed gain is 95% as good and the blast radius is 5% as bad.

 **Warning.** "Dangerously skip permissions" is a real thing said by adults who picked the name on purpose. Treat it that way.

2. Letting One Conversation Run All Day

You started at 9 AM on the AKI cohort SQL. You drifted to the pathology prompt. You wandered into a Power BI question. It's 4 PM and Claude has started to forget what year you said. This is **context rot**, and it's the silent killer of long sessions.

Use `/clear` between unrelated tasks. After two failed corrections, `/clear` and write a fresh prompt that includes what you just learned. A clean 30-minute session beats a tangled 4-hour one every single time.

3. Pasting an Entire CLAUDE.md from Twitter

Somebody on social media shared their 600-line `CLAUDE.md` and you copied the whole thing. Now Claude is following rules for a stack you don't use, in a language you don't write, with style preferences that don't match your team's.

Run `/init`, then **cut the result in half**. Apply the litmus test: "Would Claude make a mistake without this line?" If no, delete it. You have ~150–200 lines of effective compliance budget. Don't burn it on inherited dogma.

4. Describing Bugs Instead of Pasting Them

"There's some kind of error with the date column, I think it's not parsing right, the second row looks weird."

Stop. Paste the actual error log. Paste the actual row. Use the `!` prefix:

```
!python scripts/load_vitals.py 2>&1 | head -50
```

Claude doesn't need your interpretation. It needs the data. Abstractions hide the symptom.

5. Approving Every Single `npm install` Forever

Six months in, you're still hitting the approve button on `npm test` like a pigeon at Skinner's lab. Every approval interruption costs you flow.

Allowlist the safe stuff in `.claude/settings.json`: `npm test`, `npm run lint`, `pytest`, `pre-commit run`, the `gh` CLI, `git status`, `git diff`. Keep the gates up for `git push`, `npm install`, anything that touches the internet, anything that deletes files. The middle ground is the sweet spot.

6. Skipping Plan Mode for Multi-File Changes

You typed "refactor the AKI staging logic to use the new KDIGO 2024 thresholds" and hit enter. Claude immediately started editing three notebooks at once. Forty minutes later, the changes don't pass the eval and you can't tell which file caused the regression.

Use Plan Mode (`Shift+Tab`) for anything structural. Read the plan. Edit it with `Ctrl+G`. *Then* let Claude execute. The 90 seconds you spend reviewing the plan saves you the 90 minutes of un-doing.

 **Remember.** "It went and spent 20 minutes confidently solving the wrong problem" is the most common Claude Code failure mode. Plan Mode is the antidote.

7. Not Reading the Diff Before You Commit

Claude is fast. The default rhythm is "describe → wait → commit → next." Somewhere in that loop, you stopped looking at the actual code changes.

Always read the diff. `git diff --stat` first to see what changed, then `git diff` on the suspicious files. Claude is good. Claude is not infallible. Boris Cherny says every single line of his last 30 days of work was Claude-written, but he's also the person who built the checkpoint system. He's checking.

8. Forgetting the Deny List

You'll add an allowlist on day one. You'll get to the deny list never. Then one day Claude reads your `.env` while answering a config question, the `.env` text lands in the context, and you wonder why nothing feels safe anymore.

`.claude/settings.json` deny rules for `.env`, `*.key`, `secrets/`, `data/patients/`, and anything matching `*.pem`. Claude won't even discover those files. The deny list is non-negotiable for clinical work.

9. Treating Subagents Like Magic

You read Chapter 13 and now every task becomes "delegate to a subagent." Suddenly you have five subagents running, you don't remember what each one was for, and the main thread is more confused than it was before.

Subagents are for **self-contained** tasks: "Read these 30 files and summarize what they do." If the task needs back-and-forth with you, keep it on the main thread. If you find yourself coordinating five subagents, you wanted an **agent team**, not a hub of subagents.

10. Building Without Tests, Then Asking Claude to "Make It Work"

You wrote 800 lines of cohort SQL. Nothing tests it. You hand it to Claude and say "fix the bugs." Claude makes plausible changes. You can't tell if they're correct. Two days later somebody asks why the deceased-patient count dropped 3%.

Tests are how you let Claude work autonomously. "Refactor auth. Run the test suite. Fix failures before calling it done." That one instruction produces a 2–3× quality improvement. For clinical work, the test is the eval suite. Wire it into a hook so it's automatic.

 **Tip.** If you can't test it, you can't autonomously refactor it. Investing in the eval suite is investing in your ability to use Claude Code on hard problems.

Try This

Open your current Claude Code project. Check three things, in order:

1. Does `.claude/settings.json` exist? Does it have a deny list?
2. Does `CLAUDE.md` exist? Is it under 200 lines?
3. Is there a test command in the project that Claude knows about (mentioned in `CLAUDE.md` or wired to a hook)?

If any answer is no, fix it before your next session.

Watch Out

The most expensive mistake on this list isn't a single command. It's *complacency*. The moment you stop reading the diff is the moment Claude starts shipping bugs to production with your name on the commit. Stay in the loop.

Chapter 21: Ten Slash Commands Worth Memorizing

There are dozens of slash commands. You only need ten in muscle memory.

1. `/init`

Run this once per project, on day one. It scans the codebase and drafts a starter `CLAUDE.md`. Then (per Chapter 10) you cut the result in half. Without `/init`, every new project starts cold. With it, Claude knows your stack, your test command, and your folder layout before you say a word.

2. `/clear`

The reset button. Wipes the conversation, keeps the project. Use it between unrelated tasks, after two failed corrections, and at the start of every new session that isn't a deliberate resume. The single most-used command in any serious user's day, and the one beginners use the least.

 **Remember.** A clean 30-minute conversation beats a confused 3-hour one. `/clear` is how you get there.

3. `/compact`

When you want to keep going but the context window is filling up, `/compact` summarizes everything so far and continues with the summary in place of the full transcript. Always pass an argument: `/compact focus on the schema migration decisions`. Without guidance, compaction can lose the thread.

4. `/rewind`

The "I changed my mind" command. Opens a list of every prompt from your session and lets you jump back to any point. Pair with `Esc+Esc` for the keyboard shortcut. Restore code only, conversation only, or both. This is your undo for ideas you were only 40% sure about.

5. `/model`

Switch models mid-session. `/model haiku` when you're doing trivial bulk work. `/model sonnet` for the default daily flow. `/model opus` for the hard reasoning, the architecture choice, the suspicious bug. `/model opus[1m]` if you need the 1-million-token context window. Match the model to the task and you'll halve your spend.

6. `/cost`

How much have you spent? `/cost` answers in real dollars for the current session. `/stats` adds tokens and per-model breakdown. Check this before lunch and before you go home. The number teaches you which kinds of work are expensive and which are cheap, which is half of using the tool well.

7. `/permissions`

The allowlist editor without the JSON. Type `/permissions`, get a UI, tick the boxes for the commands you trust. The result is written to `.claude/settings.json`, which you can check into git so your teammates inherit the same trust list. This is the safer alternative to `--dangerously-skip-permissions`, and the one Boris Cherny recommends.

8. `/agents`

Open the subagent manager. Create new specialized agents (a strict-TypeScript reviewer, a Haiku-based quick-search agent, a docs writer) and save them to `.claude/agents/`. Once saved, you can summon them by description: "Use the security-reviewer to check this file." The setup is 5 minutes per agent and pays off forever.

9. `/btw`

A side-question overlay. "By the way, why this approach?" You get an answer without polluting the main conversation history. Useful when you want a clarification but don't want it in the transcript that compaction will summarize. Small command, big effect on long sessions.

10. `/statusline`

Run once. Get a live HUD at the bottom of your terminal: current directory, git branch, context usage percentage, model in use. Suddenly you can see when you're at 78% of your context window without guessing. Suddenly you notice you're still on `opus` for the trivial work. Visibility pays off.

Honorable mentions

A few that didn't make the top ten but earn their keep:

- `/help` : every command, alphabetical. Use it when your brain blanks.
- `/voice` : push-to-talk. People speak more context than they type.
- `/loop` : schedule a recurring background check ("check if the deploy succeeded in 5 minutes").
- `/sandbox` : OS-level isolation for risky experiments.
- `/config` : pick your output style (Explanatory, Concise, Technical).

Try This

Make yourself a `~/.claude/commands/aidi-eval.md` file with this content:

```
---
description: Run the deceased-patient eval suite and report deltas
---
Run the eval suite at aidi_catalog.dbo.eval_runs against the frozen
deceased-patient cohort. Compare counts to yesterday's baseline. Report
each delta with the patient MRN (Optimus-encoded only) and the diff
in the structured output. Don't proceed if any case is skipped.
```

You now have `/aidi-eval` as a custom slash command in every Claude Code session, forever. Multiply this by twenty rituals and you've built your own command-line.

Watch Out

Don't memorize every slash command; you'll forget half of them by Friday. Memorize **these ten**, and learn `/help` so you can find the rest when you need them. The goal is fluency, not encyclopedic recall.

Chapter 22: Ten Things to Put in Your CLAUDE.md Today

Your `CLAUDE.md` is the single highest-payoff file in any project. These are the ten lines that earn their keep at a cancer center.

Apply the litmus test to every one of them: "Would Claude make a mistake without this?" If your project doesn't need it, skip it. But for clinical AI work at KHCC, the answer is yes more often than not.

1. Your stack, in one sentence

```
Stack: PySpark on Databricks (Unity Catalog `aidi_catalog`), PydanticAI for
structured extraction, Azure OpenAI `gpt-4.1-mini`, R + tidymodels for analysis,
Django + Postgres for dashboards.
```

Without this, Claude guesses. With this, every code suggestion lands in the right place on the first try.

2. The test command

```
Tests: `pytest tests/ -q`. Eval suite: `python aidi/eval/run_deceased_cohort.py`.
Never claim "done" without one of these passing.
```

Tests are how Claude works autonomously. If it doesn't know how to run them, it can't close the loop on its own work.

3. The PHI rule

```
PHI: Never log MRNs or names in plaintext. MRNs are Optimus-encoded,
names are Fernet-encrypted. Outputs to email/dashboards use encoded MRNs only.
Eval cohorts use the frozen deceased-patient set; never include living patients.
```

Non-negotiable. This line alone justifies every minute you spend on `CLAUDE.md`.

 **Warning.** Pair this rule with a `PreToolUse` hook that blocks any `Bash` command containing patterns that look like raw MRNs. Belt and suspenders.

4. The simplicity rule

Simplicity: minimum code that solves the problem. No CLI flags, config files, or parameterization "in case." If a senior engineer would say it's overcomplicated, simplify.

Claude defaults to over-engineering. The Anthropic Data Science team uses the explicit prompt "Why are you doing this? Try something simpler" as their override. Codify it in your `CLAUDE.md` and you won't have to say it twice a day.

5. The surgical-changes rule

Surgical changes: touch only what's needed. Don't refactor adjacent code, comments, or formatting. Don't rename existing columns in extraction tables; downstream R/dashboard code depends on them. Add new columns; don't reorder.

This is the line that prevents Claude from "improving" the code review you didn't ask it to improve.

6. The fail-loud rule

Fail loud. "Pipeline completed" is wrong if any rows were skipped silently. "Eval passed" is wrong if any cases were skipped. Always surface skipped rows, null fields, parse failures, partial successes.

Silent failures kill clinical pipelines. A patient missing from the AKI alert run because of a join error is worse than the pipeline crashing. Make this rule explicit.

7. The eval gate

Verification gate: no prompt or extraction logic change ships without an eval run against the frozen deceased-patient cohort. The eval is the test.

Without this, prompt regressions ship silently. With this, every change has a binary gate.

8. Tool-calling quirks

Tool calls: run ``pytest``, not ``python -m pytest``. Don't ``cd`` unnecessarily; use the full path. Prefer ``gh`` CLI over the GitHub MCP server (more context-efficient). Don't skip pre-commit hooks.

These look petty in isolation. They are the difference between a smooth session and a chaotic one. The Anthropic RL Engineering team has the exact tic about `pytest` written in their `CLAUDE.md`. Yours probably needs its own.

9. The token-budget rule

```
Token budget per task: 4,000. Per session: 30,000. If approaching, summarize
and start fresh; don't push through. Log token usage per call in extraction
loops; fail loud when an outlier note blows the budget.
```

Karpathy's Rule 6, slightly KHCC-flavored. Without a budget, a single 50-page discharge summary quietly eats your daily quota.

10. Read-before-write

```
Before adding to a file: read the file's exports, the immediate caller, and
shared utilities. Before adding a column to an extraction table, check what
currently consumes it. "Looks orthogonal to me" is the most dangerous phrase
in this codebase.
```

The line that prevents 90% of regressions in mature pipelines.

The starter CLAUDE.md, in full

Copy this into any new KHCC project and tune from there:

```
# CLAUDE.md

## Stack
PySpark + Databricks (`aidi_catalog`), PydanticAI for extraction,
Azure OpenAI `gpt-4.1-mini`, R + tidymodels for analysis,
Django + Postgres for apps.

## Run / test
- Tests: `pytest tests/ -q`
- Eval: `python aidi/eval/run_deceased_cohort.py`
- Lint: `pre-commit run --all-files`

## Behavior rules
1. Think before coding: verify schemas/codes against actual tables; don't guess.
2. Simplicity first: minimum code that solves the problem.
3. Surgical changes: don't touch adjacent code.
4. Goal-driven: define success criteria, loop until verified.
5. LLM only for judgment: KDIGO staging is rules, compute it.
6. Token budgets: 4k/task, 30k/session.
7. Surface conflicts: don't blend contradictory patterns.
8. Read before write.
9. Tests encode intent: eval cases test clinical meaning.
10. Checkpoint after every significant step.
11. Match codebase conventions even if you disagree.
12. Fail loud: surface every skipped row, null, partial success.

## PHI rules
- MRNs: Optimus-encoded only in any output.
- Names: Fernet-encrypted at rest, never in logs.
- Eval cohort: frozen deceased-patient set only.

## Tool quirks
- `pytest`, not `python -m pytest`
- `gh` CLI > GitHub MCP
- Don't skip pre-commit hooks
```

That's it. Sixty lines. Under the 150-line compliance budget. Every line earns its place.

Try This

Open whatever project you're working on right now. Copy the starter `CLAUDE.md` above into it. Then spend 15 minutes adapting each line to your actual situation. Delete what doesn't apply. Add three project-specific quirks you've already noticed Claude getting wrong.

Watch Out

`CLAUDE.md` is a living file. The Anthropic Data Infra team ends every session by asking Claude to summarize the session and suggest improvements to `CLAUDE.md`. That feedback loop is what turns a starter file into a *good* file over six months. Do it.

APPENDICES

Appendix A: The Claude Code Cheat Sheet

Print this. Tape it to your monitor.

Open / close / reset

What	How
Open Claude Code	<code>claude</code>
Close	<code>Ctrl+C</code> twice
Clear conversation	<code>/clear</code>
Resume last session	<code>claude --resume</code>
Help (list commands)	<code>/help</code>

The ten essential slash commands

Command	What it does
<code>/init</code>	Generate starter <code>CLAUDE.md</code> (then cut in half)
<code>/clear</code>	Wipe conversation, keep project
<code>/compact <focus></code>	Summarize history, keep going
<code>/rewind</code>	Restore code or conversation to any prior point
<code>/model <name></code>	Switch model mid-session (<code>haiku</code> / <code>sonnet</code> / <code>opus</code> / <code>opus[1m]</code>)
<code>/cost</code>	Session spend in dollars
<code>/permissions</code>	Edit allow/deny lists
<code>/agents</code>	Manage subagents
<code>/btw <question></code>	Side-question without polluting history
<code>/statusline</code>	Live HUD at bottom of terminal

Keystrokes

Key	What it does
<code>Shift+Tab</code>	Enter Plan Mode
<code>Ctrl+G</code>	Edit Claude's plan in your editor
<code>Esc</code>	Stop Claude mid-action
<code>Esc Esc</code>	Open <code>/rewind</code> menu
<code>Ctrl+S</code>	Stash a draft prompt
<code>Ctrl+B</code>	Background a long-running task
<code>!cmd</code>	Run bash, land output in context
<code>@path/to/file</code>	Reference a file directly
<code>Cmd+V</code> / <code>Ctrl+V</code>	Paste a screenshot

Models

Model	When
Haiku	Bulk classification, simple file ops
Sonnet	Daily default, most coding
Opus	Hard reasoning, suspicious bugs, architecture
Opus 4.6 with ultrathink	Adaptive deep reasoning on hard problems
opus[1m]	When you need the 1M-token context window

The 8 built-in tools

Read Write Edit Bash Glob Grep WebSearch WebFetch

File layout

```
~/.claude/                                # personal, global
├── CLAUDE.md                             # your global preferences
├── settings.json                         # global permissions + hooks
├── commands/                             # personal slash commands
└── agents/                               # personal subagents

<your-project>/
├── CLAUDE.md                             # project preferences
├── .claude/
│   ├── settings.json                     # project allow/deny + hooks
│   ├── commands/                         # project slash commands
│   ├── agents/                           # project subagents
│   ├── skills/<name>/SKILL.md            # project skills
│   └── rules/                             # conditional rules (path frontmatter)
```

Permissions one-liner (`.claude/settings.json`)

```
{
  "permissions": {
    "allow": [
      "Read(*)",
      "Bash(npm test)",
      "Bash(pytest *)",
      "Bash(pre-commit run *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(gh pr *)"
    ],
    "deny": [
      "Read(.env)",
      "Read(secrets/**)",
      "Read(data/patients/**)",
      "Read(*.key)",
      "Read(*.pem)"
    ]
  }
}
```

Auto-format hook (`.claude/settings.json`)

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "npx prettier --write \"$CLAUDE_FILE_PATH\" 2>/dev/null || true"
      }]
    }]
  }
}
```

Headless / scripting

```
# One-shot, no prompts
claude -p "fix the failing tests" --allowedTools Read,Edit,Bash

# Pipe stuff in
cat error.log | claude -p "explain this error and suggest a fix"

# Parallel worktrees
claude --worktree feature-auth
claude --worktree bugfix-login
claude --worktree refactor-api
```

Pricing (as of writing)

Plan	Cost	When
Pro	\$20/mo	Light daily use
Max 5×	\$100/mo	Regular dev work
Max 20×	\$200/mo	Heavy power-user, long sessions
API pay-as-you-go	Per-token	Light + irregular use, or scripted batches

The starter `CLAUDE.md`

```
# CLAUDE.md
## Stack
<your stack in one sentence>

## Run / test
- Tests: <command>
- Eval: <command>
- Lint: <command>

## Behavior rules
1. Think before coding: verify, don't guess.
2. Simplicity first: minimum code that solves the problem.
3. Surgical changes: don't touch adjacent code.
4. Tests encode intent.
5. Fail loud: surface every skipped row, null, partial success.

## PHI rules (KHCC)
- MRNs: Optimus-encoded only in outputs.
- Names: Fernet-encrypted, never in logs.
- Eval cohort: frozen deceased-patient set only.
```

Daily flow

1. `claude` (or `cc` if you've configured the alias)
2. `/statusline` (one-time per laptop)
3. `Shift+Tab` for anything bigger than a one-liner
4. Read the plan, `Ctrl+G` to edit if needed
5. Watch the diff before approving each change
6. `/clear` between unrelated tasks
7. `/cost` before lunch, `/cost` before you leave

Appendix B: Build Your Own Agent with the Claude Agent SDK

The rest of this book treated Claude Code as a tool you talk to. This appendix treats it as a library you build with.

If you've used Claude Code, you've seen what an AI agent can actually do: read files, run commands, edit code, figure out the steps to accomplish a task. The **Claude Agent SDK** is that same engine, exposed as a library you can point at whatever problem you want. You get the agent loop, the built-in tools, context management — basically everything you'd otherwise have to build yourself.

This appendix walks you through building a small agent end to end. Pick it up when you've outgrown the interactive workflow and want to embed a Claude agent in a Python script, a CI job, or a Django backend.

📌 **Remember.** Agent SDK = Claude Code minus the terminal UI, plus a programmatic API. Same loop, same tools, same context system. You're not learning a new model; you're learning a new front door.

When to reach for the SDK

You probably want the SDK if:

- You're building a service (a webhook, a CI bot, a scheduled job) that calls Claude.
- You want **structured output** that downstream code can parse (JSON Schema enforced).
- You need **custom tools** (your own database, your own API) that Claude can call directly.
- You want to run Claude **inside a Django view** or behind an Azure Function.

You probably don't need it if you're sitting at a keyboard doing daily dev work; that's what the CLI is for.

Install

```
# Node.js 18+
npm install -g @anthropic-ai/claude-code # the runtime
npm install @anthropic-ai/claude-agent-sdk # the library
export ANTHROPIC_API_KEY=sk-ant-...
```

Python bindings exist too (`pip install claude-agent-sdk`). The patterns below are TypeScript because that's what Anthropic ships first, but they translate one-to-one.

Your first agent

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "What files are in this directory?",
  options: {
    model: "opus",
    allowedTools: ["Glob", "Read"],
    maxTurns: 250
  }
})) {
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if ("text" in block) console.log(block.text);
    }
  }
  if (message.type === "result") {
    console.log("Done:", message.subtype);
  }
}
```

That's the whole loop. `query()` returns an async generator that streams messages: assistant text, tool calls, system events, the final result. The SDK handles the agent loop: call the model, check if it wants a tool, execute it, feed the result back, repeat until done.

Message types

```
case "system":
  // Session init: capture session_id for resuming later
  if (message.subtype === "init") {
    sessionId = message.session_id;
  }
  break;

case "assistant":
  // Claude's text and tool calls
  break;

case "result":
  // Final outcome: message.subtype is "success" or an error type
  // message.total_cost_usd is the spend
  // message.usage has token counts
  break;
```


Structured output

For programmatic use, force JSON Schema:

```
const reviewSchema = {
  type: "object",
  properties: {
    issues: {
      type: "array",
      items: {
        type: "object",
        properties: {
          severity: { enum: ["low", "medium", "high", "critical"] },
          category: { enum: ["bug", "security", "performance", "style"] },
          file: { type: "string" },
          line: { type: "number" },
          description: { type: "string" },
          suggestion: { type: "string" }
        },
        required: ["severity", "category", "file", "description"]
      },
    },
    summary: { type: "string" },
    overallScore: { type: "number" }
  }
};

for await (const message of query({
  prompt: "Review the code in ./src and identify all issues.",
  options: {
    model: "opus",
    allowedTools: ["Read", "Glob", "Grep"],
    outputFormat: { type: "json_schema", schema: reviewSchema }
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    const review = message.structured_output;
    // review.issues is now a typed array; feed it to Slack, a DB, anything.
  }
}
```

This is the unlock for production use. The result is parseable. You don't regex-match on natural language anymore.

Permission modes

```
options: {
  permissionMode: "default",           // Prompt for approval (interactive only)
  // OR
  permissionMode: "acceptEdits",       // Auto-approve file edits
  // OR
  permissionMode: "bypassPermissions" // No prompts; use in CI only
}
```

For fine-grained control, supply a `canUseTool` callback:

```
options: {
  canUseTool: async (toolName, input) => {
    if (["Read", "Glob", "Grep"].includes(toolName)) {
      return { behavior: "allow", updatedInput: input };
    }
    if (toolName === "Write" && input.file_path?.includes(".env")) {
      return { behavior: "deny", message: "Cannot modify .env files" };
    }
    return { behavior: "allow", updatedInput: input };
  }
}
```

⚠ Warning. `bypassPermissions` in a clinical pipeline that touches `aidi_catalog` is a fast way to lose evening sleep. Always wrap with `canUseTool` and at minimum block writes to PHI paths.

Subagents in the SDK

You can define specialists inline:

```
options: {
  allowedTools: ["Read", "Glob", "Grep", "Task"], // Task enables subagents
  agents: {
    "security-reviewer": {
      description: "Security specialist for vulnerability detection",
      prompt: "You are a security expert. Focus on injection, secrets, auth.",
      tools: ["Read", "Grep", "Glob"],
      model: "sonnet"
    },
    "test-analyzer": {
      description: "Test coverage and quality analyzer",
      prompt: "You are a testing expert. Find gaps and missing edge cases.",
      tools: ["Read", "Grep", "Glob"],
      model: "haiku" // Cheaper model for simpler analysis
    }
  }
}
```

The main agent decides when to delegate. Each subagent runs in its own context window with its own tools and (optionally) a cheaper model.

Custom tools via MCP

When the built-in tools aren't enough, expose your own via the Model Context Protocol:

```
import { tool, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";

const aidiServer = createSdkMcpServer({
  name: "aidi-db",
  version: "1.0.0",
  tools: [
    tool(
      "query_aidi",
      "Query aidi_catalog read-only. Returns up to 100 rows.",
      { sql: z.string().describe("SELECT-only SQL") },
      async (args) => {
        const rows = await runReadOnlyQuery(args.sql);
        return { content: [{ type: "text", text: JSON.stringify(rows) }] };
      }
    )
  ]
});

for await (const message of query({
  prompt: "Find all deceased patients with stage 3 AKI in 2025.",
  options: {
    model: "opus",
    mcpServers: { "aidi-db": aidiServer },
    allowedTools: ["mcp__aidi-db__query_aidi"]
  }
})) {
  // ...
}
```

You've just given Claude SQL access to `aidi_catalog` with a single read-only handler, with no tool-loop boilerplate. This is the pattern for embedding Claude into real KHCC services.

Hooks in the SDK

Same hooks as the CLI, exposed programmatically:

```
const blockDangerous: HookCallback = async (input) => {
  if (input.hook_event_name === "PreToolUse" && input.tool_name === "Bash") {
    const cmd = input.tool_input.command || "";
    if (cmd.includes("rm -rf") || cmd.includes("DROP TABLE")) {
      return {
        hookSpecificOutput: {
          hookEventName: "PreToolUse",
          permissionDecision: "deny",
          permissionDecisionReason: "Dangerous command blocked"
        }
      };
    }
  }
  return {};
};

options: {
  hooks: {
    PreToolUse: [
      { matcher: "Bash", hooks: [blockDangerous] }
    ]
  }
}
```

 **Technical Stuff.** Hooks have signatures `(input, toolUseId, { signal }) => Promise<HookResult>`. The `signal` is an `AbortSignal` for cancellation. Return `{}` to allow the operation.

Session resumption

Multi-turn conversations from a service:

```
let sessionId;
for await (const msg of query({ prompt: "Review the codebase.", options })) {
  if (msg.type === "system" && msg.subtype === "init") sessionId = msg.session_id;
}

// Later, in another HTTP request:
for await (const msg of query({
  prompt: "Now fix the most critical issue.",
  options: { ...options, resume: sessionId }
})) {
  // Claude remembers everything from the first call.
}
```

This is how you build a stateful Claude-backed service without re-sending the whole history.

Cost tracking

```
if (message.type === "result" && message.subtype === "success") {
  console.log("Cost:", message.total_cost_usd);
  console.log("Tokens:", message.usage);
  for (const [model, usage] of Object.entries(message.modelUsage)) {
    console.log(`${model}: ${usage.costUSD.toFixed(4)}`);
  }
}
```

Per-model breakdown is essential when subagents are in play. You'll discover the Haiku-based search agent is essentially free and the Opus reviewer is the entire bill.

A real KHCC pattern

A nightly extraction service:

1. Azure Function fires at 02:00.
2. Spawns a `query()` call with the AIDI extraction prompt, JSON Schema output, and a `canUseTool` callback that blocks writes outside `extracts/`.
3. Custom MCP server lets Claude query schema from `AIDI-DB` read-only.
4. Subagent `security-reviewer` (Sonnet) scans the output for accidental PHI leakage.
5. `Stop` hook runs the eval suite, blocks return if anything regressed.
6. Result lands as JSON in blob storage; downstream pipeline picks it up at 02:30.

No human in the loop. Full audit trail (every tool call logged via `PreToolUse` audit hook). Total cost: under \$2 per night.

Where to go next

- **Official docs:** the SDK reference covers all message types, options, and tool schemas.
- **The CLI is the SDK with a UI.** Everything you can do interactively, you can script, so use the CLI to prototype, then port to the SDK.
- **Skills package up SDK-friendly capabilities.** A skill you write in `.claude/skills/` works equally well in the CLI and via `query()`.

Closing

You now have a complete picture. Claude Code (the CLI you've been using for 22 chapters) is one front door to the agent. The SDK is the other. Same engine, different ergonomics. Pick the one that fits the job in front of you.

The clinical AI office at KHCC will outlive any individual tool, model, or library on this list. Build with that in mind: small, testable, surgical, fail-loud. Claude Code is a force multiplier on those habits. It is not a substitute for them.

Now go ship something.